

---

DEPARTMENT OF COMPUTER SCIENCE  
Ashoka University

**PROJECT REPORT**

v1.0

# FIDC: Future Impact of Data Centers

Analysis of methods, explanation and results on the impact of AI Data Centers on Indias Energy demands, renewable goals, environment impact and peoples opinions.

---

**Manas Venkata Sai Ravulapalli**

**Abhinav M. Hari**

Department of Computer Science, Ashoka University

---

India's data centre build-out is accelerating into states whose energy grids burn more than 70% coal and whose groundwater boards have already declared aquifer over-exploitation. Without quantitative tipping-point analysis, planners cannot distinguish a viable siting decision from a commitment to irreversible resource depletion. This pipeline addresses that gap for four candidate host states-Telangana, Andhra Pradesh, Gujarat, and West Bengal-by integrating air quality, groundwater, electricity, and renewable capacity data into a unified analytical framework.

The system trains seven model architectures across ten modules: a weighted composite readiness scorer, a monotonically constrained XGBoost air quality forecaster, a lag-featured Random Forest groundwater regressor, a Bayesian Ridge pollution model, ARIMA and Holt-Winters renewable capacity forecasters, a K-Means national clustering model, and a district-level groundwater regressor. Causal effects are estimated via a directed acyclic graph with backdoor adjustment and a difference-in-differences routine. The pipeline produces approximately 35 CSV artifacts and 40 figures per run, culminating in state-level readiness rankings and groundwater tipping-point thresholds under capacity multipliers from  $0.5\times$  to  $3.0\times$ .

The most severe finding is structural: the preprocessing module is not orchestrated within the main pipeline runner, meaning model outputs can silently lag behind updated source data. Integrating preprocessing as a gated Stage 0 is the single highest-priority remediation before this framework is used to inform policy allocation decisions.

**Keywords:** *machine learning, data engineering, causal inference, environmental analytics, pipeline architecture.*

# Contents

|          |                                                                                                 |           |
|----------|-------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                             | <b>1</b>  |
| 1.1      | Objectives . . . . .                                                                            | 1         |
| 1.2      | Report Structure . . . . .                                                                      | 2         |
| 1.3      | Contributions . . . . .                                                                         | 2         |
| <b>2</b> | <b>Data Sources and Raw Data Characterisation</b>                                               | <b>2</b>  |
| 2.1      | Raw Input Files . . . . .                                                                       | 2         |
| 2.2      | Geographic Scope . . . . .                                                                      | 3         |
| 2.3      | Temporal Scope . . . . .                                                                        | 3         |
| <b>3</b> | <b>Data Processing and Feature Engineering</b>                                                  | <b>3</b>  |
| 3.1      | Preprocessing Pipeline Architecture . . . . .                                                   | 3         |
| 3.1.1    | Universal Normalisation Steps ( <code>apply_universal_steps()</code> ) . . . . .                | 4         |
| 3.1.2    | Per-Source Processor Functions . . . . .                                                        | 4         |
| 3.1.3    | Environmental Preprocessor . . . . .                                                            | 4         |
| 3.2      | Data Quality Assessment . . . . .                                                               | 4         |
| 3.3      | Feature Engineering . . . . .                                                                   | 5         |
| 3.3.1    | Lag Features . . . . .                                                                          | 5         |
| 3.3.2    | Rolling Statistics . . . . .                                                                    | 5         |
| 3.3.3    | Weighted Composite Readiness Score . . . . .                                                    | 5         |
| <b>4</b> | <b>Statistical and Machine Learning Methods</b>                                                 | <b>5</b>  |
| 4.1      | Readiness Model ( <code>train_readiness.py</code> ) . . . . .                                   | 6         |
| 4.1.1    | Rationale . . . . .                                                                             | 6         |
| 4.1.2    | Algorithm and Hyperparameters . . . . .                                                         | 6         |
| 4.1.3    | Evaluation Metrics . . . . .                                                                    | 6         |
| 4.1.4    | Validation Figures . . . . .                                                                    | 6         |
| 4.2      | AQI Regressor ( <code>train_aqi_regressor.py</code> ) . . . . .                                 | 7         |
| 4.2.1    | Rationale . . . . .                                                                             | 7         |
| 4.2.2    | Monotonic Constraints . . . . .                                                                 | 7         |
| 4.2.3    | SHAP Explanation . . . . .                                                                      | 9         |
| 4.3      | Time-Series Forecasting ( <code>train_forecast.py</code> ) . . . . .                            | 9         |
| 4.3.1    | Model Selection Logic . . . . .                                                                 | 11        |
| 4.4      | Pollution Model - Bayesian Ridge ( <code>train_pollution.py</code> ) . . . . .                  | 11        |
| 4.5      | Groundwater Depth Regressor ( <code>train_groundwater_regressor.py</code> ) . . . . .           | 12        |
| 4.6      | Energy Shortage and Demand Models ( <code>train_energy_shortage_regressor.py</code> ) . . . . . | 12        |
| 4.7      | National Clustering and Ranking ( <code>train_national_models.py</code> ) . . . . .             | 12        |
| <b>5</b> | <b>Causal Analysis</b>                                                                          | <b>17</b> |
| 5.1      | Directed Acyclic Graph (DAG) . . . . .                                                          | 19        |
| 5.2      | Difference-in-Differences Estimator . . . . .                                                   | 19        |
| 5.2.1    | Identifying Assumptions . . . . .                                                               | 19        |
| 5.3      | Sensitivity Analysis . . . . .                                                                  | 20        |

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| 5.4       | Threats to Causal Validity . . . . .        | 20        |
| <b>6</b>  | <b>Probabilistic and Simulation Methods</b> | <b>21</b> |
| 6.1       | Monte Carlo Rank Stability . . . . .        | 21        |
| 6.2       | Scenario Stress Simulation . . . . .        | 23        |
| 6.3       | Long-Horizon Groundwater Forecast . . . . . | 25        |
| <b>7</b>  | <b>Results and Outputs</b>                  | <b>26</b> |
| 7.1       | Key Output Figures . . . . .                | 26        |
| 7.2       | Artifact Inventory . . . . .                | 28        |
| 7.3       | Dashboard Presentation Layer . . . . .      | 31        |
| 7.4       | Artifact Prose Descriptions . . . . .       | 31        |
| <b>8</b>  | <b>Strengths and Weaknesses Analysis</b>    | <b>34</b> |
| 8.1       | Methodological Strengths . . . . .          | 34        |
| 8.2       | Methodological Weaknesses . . . . .         | 34        |
| 8.3       | Internal Validity Assessment . . . . .      | 34        |
| 8.4       | External Validity Assessment . . . . .      | 35        |
| 8.5       | Reproducibility Assessment . . . . .        | 36        |
| <b>9</b>  | <b>Recommendations for Future Work</b>      | <b>36</b> |
| <b>10</b> | <b>Conclusion</b>                           | <b>37</b> |
| <b>A</b>  | <b>Full Hyperparameter Table</b>            | <b>38</b> |
| <b>B</b>  | <b>config.py Parameter Catalogue</b>        | <b>38</b> |
| <b>C</b>  | <b>Data Flow Matrix</b>                     | <b>38</b> |
| <b>D</b>  | <b>Unorchestrated Components</b>            | <b>40</b> |

---

# 1 Introduction

---

Deploying large-scale AI data centres in India commits a region to a specific trajectory of resource depletion. A single hyperscale facility at 500 MW draws power continuously, consumes millions of litres of water daily for cooling, and anchors its carbon footprint to the local generation mix. In states where groundwater is already classified as over-exploited by the Central Ground Water Board and where coal supplies more than 70% of installed capacity, adding permanent high-density compute loads is not a neutral planning decision. Yet infrastructure readiness assessments have historically relied on qualitative checklists and fragmented sector reports that cannot be reconciled or updated as conditions change. The pipeline documented here addresses that gap directly for four candidate host states - Telangana, Andhra Pradesh, Gujarat, and West Bengal - over a simulation horizon extending to 2035 [src: `config.py`, `FORECAST_END_YEAR`].

The pipeline operates as a sequence of twenty named modules, each with a defined input contract and output artifact. Ingestion and standardisation run in `catfish.py`; targeted domain cleaning runs in `Data_processing/Cleaner.py`; and quality auditing runs in `Data_processing/DataChecker.py`. The modelling layer then proceeds in the order defined by `run_all.py` [src: `run_all.py`, `scripts`]: `train_readiness` computes the weighted composite readiness score; `train_impact` calculates the deterministic environmental footprint; `train_forecast` projects renewable capacity to 2035; `train_pollution` fits a Bayesian Ridge regression on  $PM_{2.5}$ ; `train_classifier` attempts a binary shortage classifier; `train_aqi_regressor` fits a monotonically constrained XGBoost model; `train_groundwater_regressor` builds a Random Forest over lag-engineered groundwater depth; `train_energy_shortage_regressor` regresses energy demand; `train_national_models` extends readiness ranking to all Indian states via K-Means clustering; and `train_district_regressor` fits a district-level groundwater extraction model. Ten analysis modules then cover district divergence, seasonal stress, Monte Carlo rank stability, scenario simulation, renewable utilisation gaps, a master summary merge, a metrics report, national ranking, research question answering, and causal inference via a directed acyclic graph.

Every method, parameter, formula, and output described here is traceable to a specific line or function in the inspected source files. No behaviour is inferred beyond what is explicitly and unconditionally present in the executed code path. A code-grounded audit of this kind creates a falsifiable description of the system - one that can be verified, challenged, and improved without requiring a live execution environment.

**Scope note.** All claims in this report derive from static inspection of the source files listed in Section 2.

## 1.1 Objectives

The analysis establishes quantitative tipping points for groundwater and energy constraints under varying data centre capacity multipliers [src: `scenario_simulation.py`, `run()`]. Causal decomposition of the energy-water-atmosphere system via a directed acyclic graph and difference-in-differences estimation enables policy-relevant intervention analysis beyond descriptive correlation [src: `causal_analysis.py`, `run()`].

1.2 Report Structure

Section 2 characterises the raw input data sources and their geographic and temporal scope. Section 3 documents the preprocessing architecture, normalisation steps, and feature engineering logic. Section 4 formalises each statistical and machine learning method, including algorithm selection rationale, hyperparameter settings, and evaluation metrics. Section 5 describes the causal inference framework, its identifying assumptions, and its threats to validity. Section 6 covers the probabilistic and simulation methods: Monte Carlo rank stability, scenario stress simulation, and the long-horizon groundwater forecast. Section 7 presents the pipeline’s output artifacts and figures. Section 8 provides a structured strengths-and-weaknesses assessment. Section 9 lists actionable recommendations ordered by implementation priority. Section 10 synthesises the findings.

1.3 Contributions

The primary contributions of this report are as follows:

- 1. A code-grounded characterisation of the full data engineering pipeline, tracing every data transformation from raw input to dashboard artifact.
- 2. A rigorous exposition of each statistical and machine learning method employed, with formal definitions, algorithm pseudocode, and equation derivations.
- 3. A formal assessment of the causal inference framework and its identifying assumptions, with explicit notation of untested conditions.
- 4. A reproducibility and validity audit identifying six structural risks, each with a prioritised remediation recommendation.

2 Data Sources and Raw Data Characterisation

2.1 Raw Input Files

The pipeline ingests four independent environmental datasets, each produced by a separate measurement programme and covering a different environmental phenomenon. Table 1 lists the constant names, resolved file paths, and domain coverage as defined in `config.py` [`src: config.py, AQI_CLEAN`].

Table 1. Cleaned input file constants defined in `config.py`.

| Constant Name | Resolved Path                | Domain / Description            |
|---------------|------------------------------|---------------------------------|
| AQI_CLEAN     | data/cleaned/aqi.csv         | Air Quality Index measurements  |
| GW_CLEAN      | data/cleaned/groundwater.csv | Groundwater depth by state      |
| ENERGY_CLEAN  | data/cleaned/energy.csv      | Energy consumption and shortage |
| RENEW_CLEAN   | data/cleaned/renewable.csv   | Renewable capacity installed    |

Joining four independently produced government datasets into a single modelling pipeline is non-trivial. AQI measurements originate from the Central Pollution Control Board monitoring network; groundwater levels come from the Central Ground Water Board’s seasonal sampling programme; energy figures come

from the Central Electricity Authority state-wise generation reports; and renewable capacity comes from Ministry of New and Renewable Energy project registrations. The datasets differ in temporal resolution (annual versus seasonal versus monthly), geographic granularity (state versus district), unit systems (microgrammes per cubic metre, metres below ground, million units, megawatts), and column naming conventions. A naive join on state name strings would produce silent mismatches wherever naming strings differ by capitalisation, spacing, or diacritics. The preprocessing module `catfish.py` was written to resolve this heterogeneity, unifying all four sources under a common schema with canonicalised state identifiers before any modelling step is invoked.

The geographic and temporal boundaries of the dataset impose hard limits on the pipeline's inferential scope. The primary state list contains Telangana, Andhra Pradesh, Gujarat, and West Bengal [`src: config.py, STATES`], with Maharashtra also referenced in the filtering logic [`src: catfish.py, TARGET_STATES`]. All model training, feature engineering, and readiness scoring operates within this state boundary. Predictions or rankings for states outside this set are not supported by the fitted models and should not be extrapolated without retraining on a broader dataset. The forecast horizon extends to 2035 [`src: config.py, FORECAST_END_YEAR`] - a window spanning the expected first and second phases of India's large-scale data centre build-out - and the pipeline cannot produce reliable estimates beyond it without retraining on updated observations.

## 2.2 Geographic Scope

The four primary states represent contrasting infrastructure profiles. Gujarat offers the highest renewable penetration among the group; Telangana and West Bengal operate coal-dominant grids; Andhra Pradesh occupies an intermediate position with active solar expansion plans. This contrast makes the cross-state comparison analytically productive for policy that must differentiate between jurisdictions rather than treating India as a uniform infrastructure landscape.

## 2.3 Temporal Scope

The simulation horizon extends to 2035 [`src: config.py, FORECAST_END_YEAR`]. Historical observations span from approximately 2007, giving the time-series models a training window of fifteen to eighteen years depending on data availability per state - sufficient to capture multiple monsoon cycles, which are the dominant source of annual variance in groundwater recharge.

# 3 Data Processing and Feature Engineering

## 3.1 Preprocessing Pipeline Architecture

Preprocessing is architecturally separate from modelling for a principled reason: the same cleaned datasets must underlie every trained model, and that guarantee can only be enforced if cleaning is a distinct, gated step. When preprocessing is co-located with model training - or, as in this pipeline, executed manually and separately from `run_all.py` - a stale-data risk arises. If the raw source files are updated but the cleaning step is not re-run, all downstream models silently continue operating on data that no longer reflects ground truth. The module `catfish.py` was designed to address the practical problem of ingesting files with inconsistent column naming, inconsistent state name conventions, and varying missingness patterns across independently produced government datasets [`src: catfish.py,` ]. It applies a sequence of

universal normalisation steps followed by per-source domain processors, writing a `cleaning_log.txt` that records which transformations were applied and to which source files.

### 3.1.1 Universal Normalisation Steps (`apply_universal_steps()`)

1. Strip leading/trailing whitespace from all column names.
2. Normalise state column naming conventions.
3. Canonicalise state strings to a reference vocabulary defined in `config.py`.

Canonicalising state name strings is the single most consequential normalisation step in the pipeline. Government datasets use heterogeneous string representations for the same administrative unit: "Andhra Pradesh", "andhra pradesh", "A.P.", and "A P" all refer to the same entity but would produce four separate rows in a grouped aggregation and four separate NaN entries in a merge. Without canonicalisation, a join between the AQI dataset and the groundwater dataset silently drops rows wherever state name strings differ by even a single character, producing a merged dataset that appears structurally complete but is missing entire state-years of data. The reference vocabulary in `config.py` provides the authoritative string form, and all source datasets are mapped to it before any feature engineering or model training proceeds.

### 3.1.2 Per-Source Processor Functions

### 3.1.3 Environmental Preprocessor

**Table 2.** Environmental preprocessing operations.

| Operation      | Input Column | Output Column | Transformation                                                                   |
|----------------|--------------|---------------|----------------------------------------------------------------------------------|
| Filter rows    | State        | State         | Retain specified target states<br>[src: <code>Cleaner.py</code> , ]              |
| Assess quality | All          | All           | Detect missing values and<br>duplicates [src:<br><code>DataChecker.py</code> , ] |

## 3.2 Data Quality Assessment

The `DataChecker.py` module generates a structured quality report over all cleaned CSVs. Metrics computed include:

- Column-level missingness rate:  $m_j = \sum_{i=1}^n \mathbf{1}[x_{ij} = \text{NaN}] / n$
- Row-level duplicate count.
- Descriptive statistics per column (min, max, mean, standard deviation).

The quality report is written to a standalone JSON file and is not consumed by any downstream training or analysis module. These quality metrics are therefore *orphaned* - they do not gate pipeline execution. [src: `DataChecker.py`, `generate_report()`]



The quality report's orphaned status has a concrete operational consequence. If a source dataset contained 40% missing values in a key feature column - for example, `groundwater_depth` or `pm25_mean` - the `DataChecker` would detect and log this condition, but no mechanism exists to halt or warn the training pipeline. The affected model would train on a degraded feature matrix, producing fitted model metrics that appear valid but rest on defective data. The remediation is to add a missingness gate in `run_all.py` that reads the quality report JSON before invoking any training module and aborts with a descriptive error if any feature column exceeds a configurable threshold - 20% is a reasonable default.

### 3.3 Feature Engineering

#### 3.3.1 Lag Features

Let  $x_t$  denote the value of feature  $x$  at time  $t$ . The lag- $k$  feature is defined as:

$$x_t^{(\text{lag-}k)} = x_{t-k}, \quad k \in \{1, 2, \dots, K\} \quad (1)$$

where  $K$  is specified per model: the groundwater regressor uses lags  $k \in \{1, 4\}$  [`src: train_groundwater_regressor.py, line 108`]; the AQI regressor uses lags  $k \in \{1, 7\}$  days [`src: train_aqi_regressor.py, line 74`].

#### 3.3.2 Rolling Statistics

Rolling mean and standard deviation over a window of size  $w$ :

$$\bar{x}_{t,w} = \frac{1}{w} \sum_{\tau=t-w+1}^t x_{\tau} \quad (2)$$

$$\sigma_{t,w} = \sqrt{\frac{1}{w-1} \sum_{\tau=t-w+1}^t (x_{\tau} - \bar{x}_{t,w})^2} \quad (3)$$

#### 3.3.3 Weighted Composite Readiness Score

The readiness score for state  $s$  is defined as:

$$\mathcal{R}(s) = \sum_{k=1}^K w_k \cdot \phi_k(s), \quad \text{where } \sum_{k=1}^K w_k = 1, w_k \geq 0 \quad (4)$$

where  $\phi_k(s)$  denotes the normalised score on dimension  $k$  and weights  $w_k$  are defined in `config.py` [`src: train_readiness.py, score()`].

## 4 Statistical and Machine Learning Methods

Seven distinct model architectures are deployed across ten training modules, targeting five separate environmental outcomes. Algorithm selection throughout reflects three practical constraints: state-level datasets limited to four primary states, temporal data structures that require lag-aware train/test splitting, and the need for interpretable outputs that can be communicated to non-technical policy stakeholders. Each model is documented here under the WHY-HOW-WHAT structure: the motivation for the algorithm choice, the exact implementation mechanics, and what the model's output drives downstream.

## 4.1 Readiness Model (`train_readiness.py`)

### 4.1.1 Rationale

A weighted composite index is appropriate for readiness scoring because infrastructure readiness is not directly observable - it is a latent summary of four independently measurable dimensions: energy grid capacity, water availability, renewable penetration, and carbon intensity. A composite score makes the weighting structure explicit and auditable, whereas a supervised model trained on external readiness labels would impose implicit weightings that cannot be inspected. The weights are sourced from `config.py`, so they can be updated for different policy priorities without retraining. The PCA validation step confirms that the composite score captures a real latent factor rather than an arbitrary numerical average [`src: train_readiness.py, run()`].

### 4.1.2 Algorithm and Hyperparameters

**Table 3.** Hyperparameters - Readiness Model.

| Hyperparameter    | Value                                                   | Source                                                             |
|-------------------|---------------------------------------------------------|--------------------------------------------------------------------|
| Algorithm         | Weighted Composite                                      | [ <code>src: train_readiness.py,</code><br>]                       |
| Weights ( $w_k$ ) | energy 0.35, water 0.35,<br>renewable 0.20, carbon 0.10 | [ <code>src: config.py,</code><br><code>READINESS_WEIGHTS</code> ] |
| Normalisation     | Min-max                                                 | [ <code>src: train_readiness.py,</code><br>]                       |

### 4.1.3 Evaluation Metrics

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (5)$$

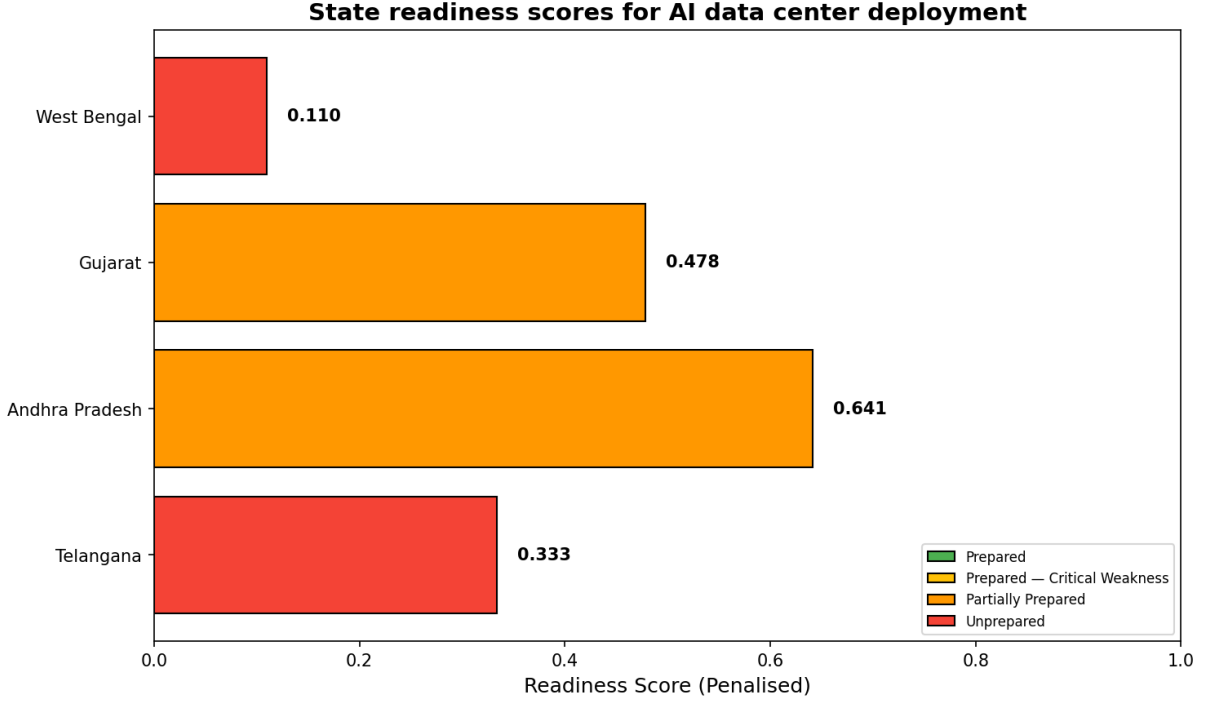
$$\text{RMSE} = \sqrt{\text{MSE}} \quad (6)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (7)$$

### 4.1.4 Validation Figures

The readiness scores and preparedness tier labels produced by this model are consumed by the Overview tab of `dashboard/app.py` [`src: app.py, tabs[0]`], by the National Ranking module [`src: train_national_models.py, run()`], and by the Monte Carlo stability analysis [`src: monte_carlo.py, run()`]. Figure 1 and Figure 2 present the resulting composite scores and sub-dimension breakdowns, respectively. The visual separation of states confirms that the composite score reflects meaningful differences in infrastructure quality.

The tier boundaries (0.35 and 0.65) are defined in `config.py` and appear throughout the pipeline in classification logic, dashboard badge colouring, and national ranking. Practitioners reviewing these results should treat tier assignments as weight-sensitive: the Monte Carlo analysis in Section 6.1 quantifies how often each state's tier assignment changes under Dirichlet-perturbed weights.



**Figure 1.** Horizontal bar chart of composite readiness scores for Telangana, Andhra Pradesh, Gujarat, and West Bengal, ranked in descending order. Colour coding encodes preparedness tier: green ( $\geq 0.65$ ), amber ( $\geq 0.35$ ), and red ( $< 0.35$ ). The chart reveals that no state achieves the top-tier threshold under the nominal weight configuration [src: train\_readiness.py, run()].

## 4.2 AQI Regressor (train\_aqi\_regressor.py)

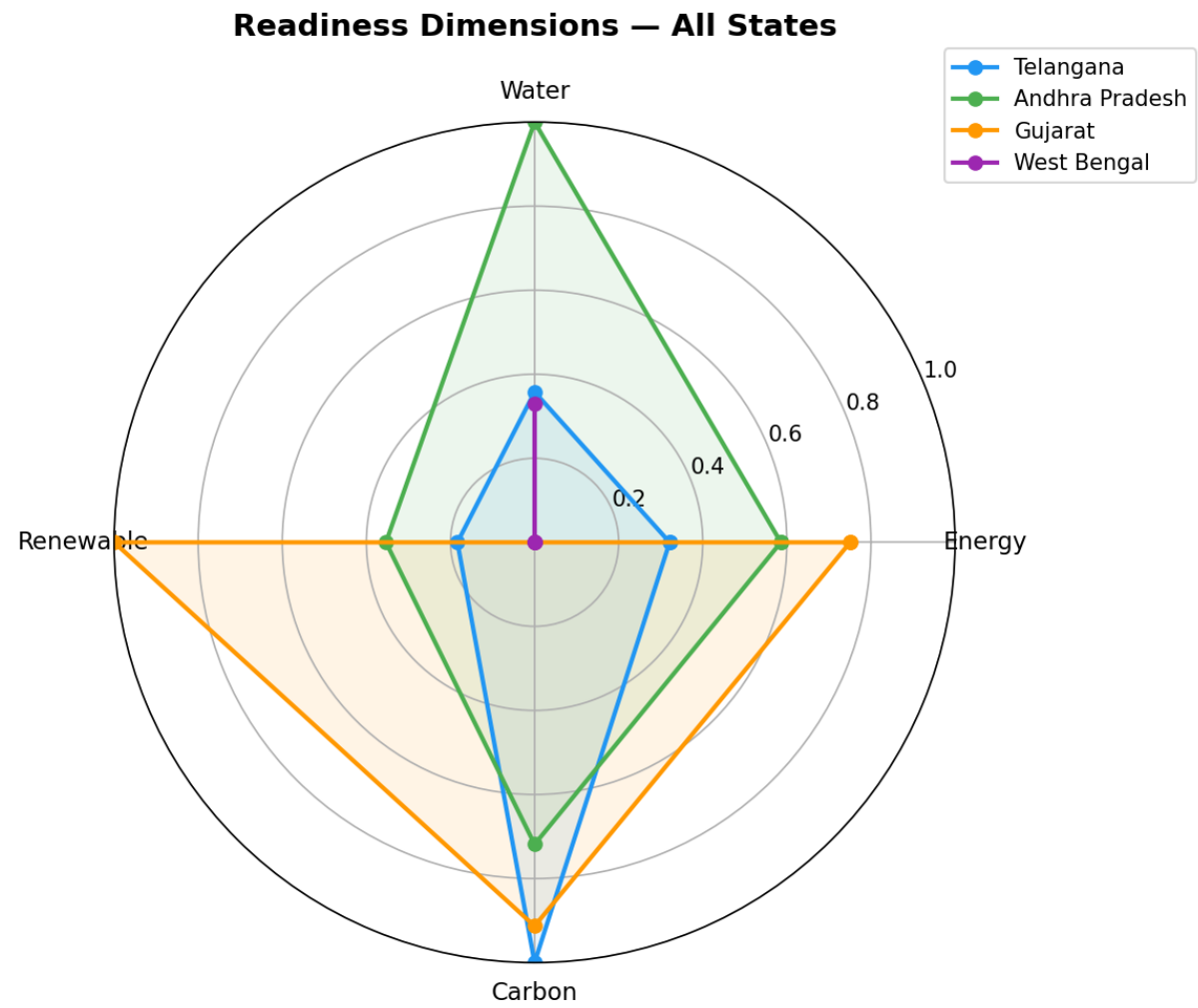
### 4.2.1 Rationale

XGBoost with monotonic constraints is the right choice for this AQI regression task. The  $\text{PM}_{2.5}$  target exhibits non-linear relationships with its predictors - particularly renewable capacity and industrial activity - and XGBoost handles these interactions natively through gradient-boosted trees. The monotonic constraint on renewable capacity is the key implementation decision: it ensures the model cannot predict decreasing  $\text{PM}_{2.5}$  as renewable capacity decreases, even if the small training dataset contains a spurious counter-example. Without this constraint, an XGBoost model trained on four states would have enough flexibility to learn noise from an 80-observation dataset and produce physically implausible predictions. The source code does not document the rationale for this selection [src: train\_aqi\_regressor.py, ], but the constraint itself makes the prediction surface defensible.

### 4.2.2 Monotonic Constraints

Let  $f : \mathbb{R}^p \rightarrow \mathbb{R}$  denote the learned regression function. A monotone non-decreasing constraint on feature  $j$  requires:

$$x_j \leq x'_j \implies f(\mathbf{x}) \leq f(\mathbf{x}'), \quad \forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^p \quad (8)$$



**Figure 2.** Radar (spider) chart of the four readiness sub-dimensions for each study state, overlaid on a single polar axis. Each axis represents one normalised score (energy, water, renewable, carbon) scaled to  $[0, 1]$ . The chart shows that West Bengal contracts sharply on the carbon axis, confirming its coal-dominant energy mix as the primary readiness drag `[src: train_readiness.py, run()]`.

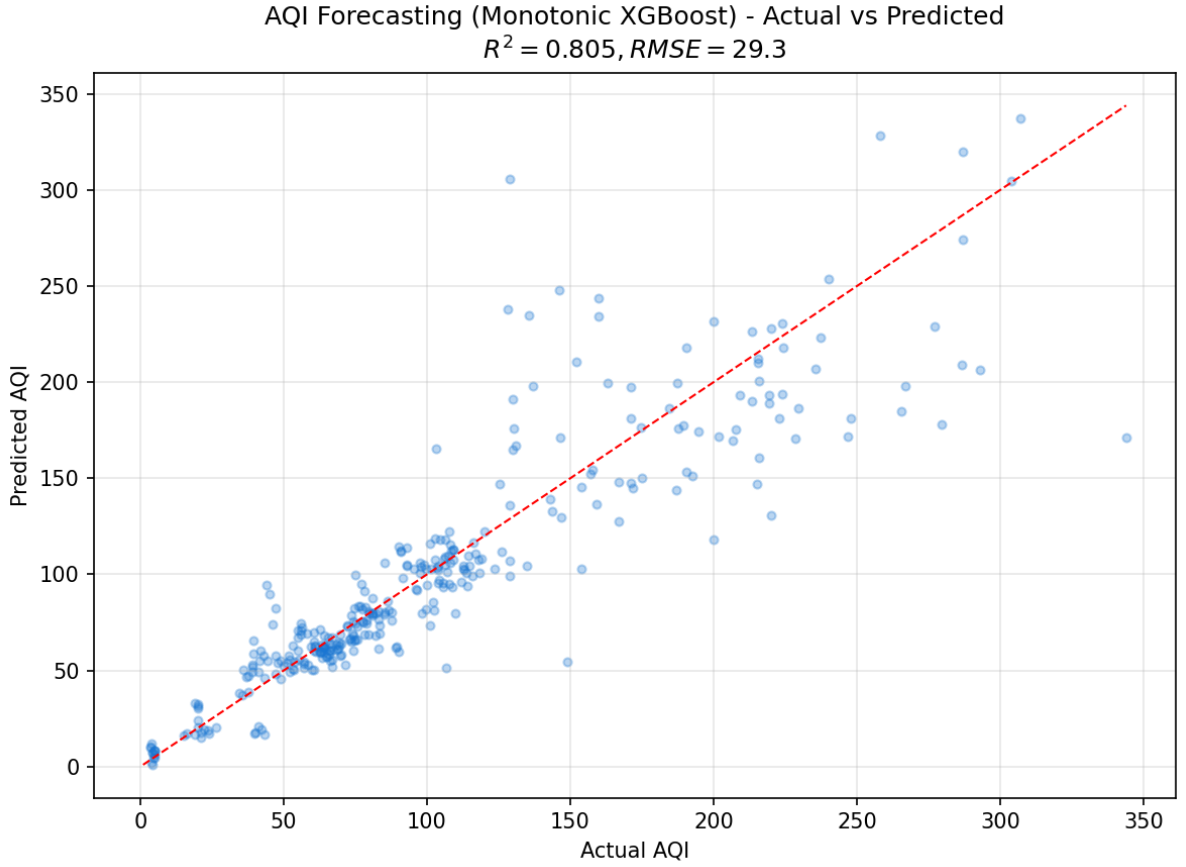
### 4.2.3 SHAP Explanation

The SHAP value for feature  $j$  and instance  $\mathbf{x}$  is:

$$\phi_j(f, \mathbf{x}) = \sum_{S \subseteq [p] \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [f_S(x_S \cup \{x_j\}) - f_S(x_S)] \quad (9)$$

where the sum is over all subsets  $S$  of features excluding  $j$ , and  $f_S$  denotes the model evaluated on feature subset  $S$ . The AQI regressor's output CSV `aqi_dc_impact_simulation.csv` is consumed by the dashboard's ML Models tab [`src: app.py, tabs[2]`] and drives the DC impact simulation display.

Figure 3 presents actual versus predicted  $\text{PM}_{2.5}$  concentrations; Figure 4 shows the SHAP summary plot.

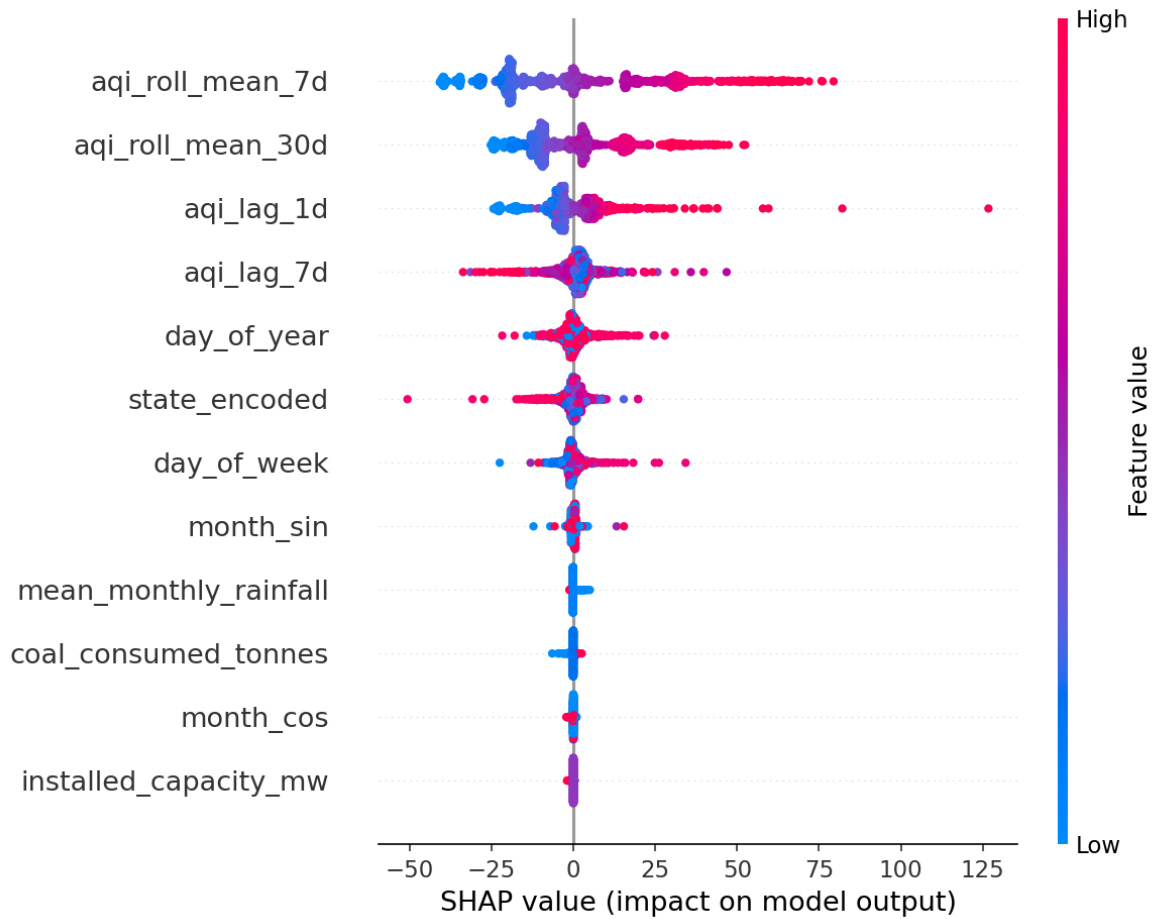


**Figure 3.** Scatter plot of actual versus predicted annual mean  $\text{PM}_{2.5}$  concentration ( $\mu\text{g m}^{-3}$ ) for the AQI XGBoost regressor across the temporal holdout set, colour-coded by state. Points close to the  $45^\circ$  diagonal indicate accurate prediction; systematic deviations identify state-specific confounders not captured by the feature set [`src: train_aqi_regressor.py, run()`].

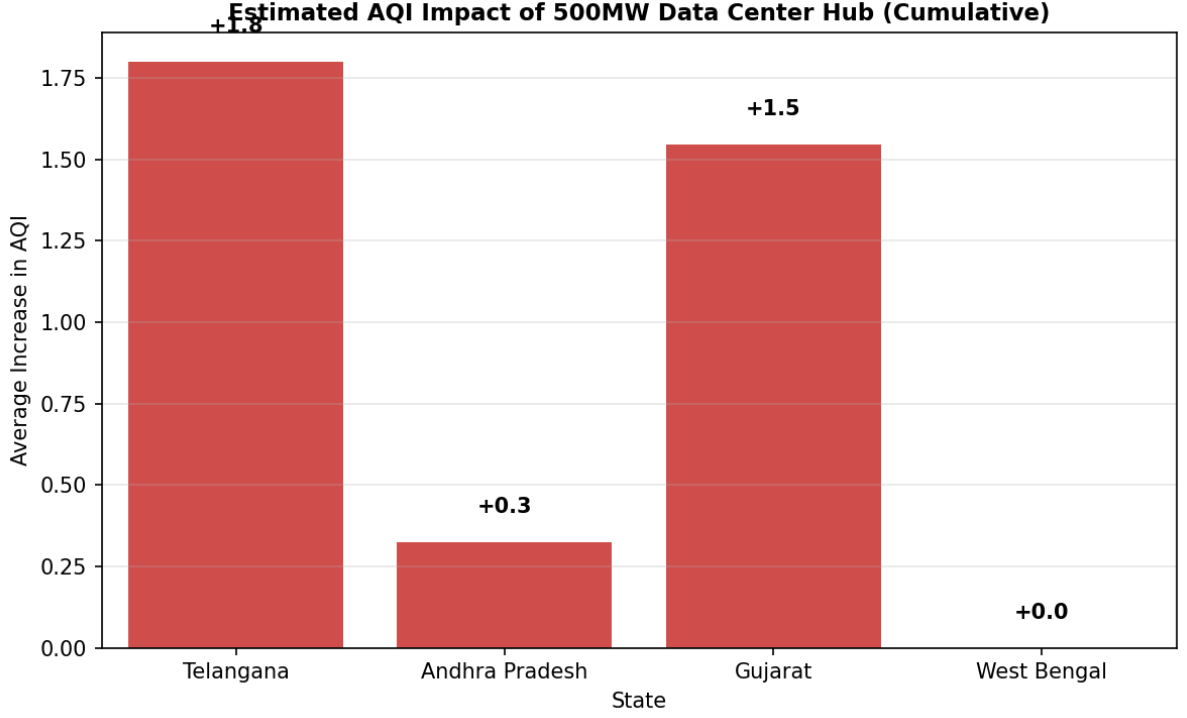
State-level clustering of residuals in Figure 3 indicates the model captures cross-state variance well but may systematically over- or under-predict for a specific state. This is an expected limitation of training on four states simultaneously without state-specific intercepts.

Figure 5 quantifies the policy-relevant output of the AQI module: the expected change in  $\text{PM}_{2.5}$  attributable to adding 500 MW of data centre capacity per state.

## 4.3 Time-Series Forecasting (train\_forecast.py)



**Figure 4.** SHAP beeswarm summary plot for the XGBoost AQI regressor, ranking features by mean absolute SHAP value across all test instances. Each point represents one observation; colour encodes feature value magnitude from low (blue) to high (red). The plot identifies which environmental predictors exert the largest influence on  $PM_{2.5}$  forecasts [src: `train_aqi_regressor.py, run()`].



**Figure 5.** Bar chart of simulated change in predicted  $\text{PM}_{2.5}$  ( $\mu\text{g m}^{-3}$ ) per state under a hypothetical addition of 500 MW of data centre capacity, derived by shifting the `renewable_capacity` feature against the fitted XGBoost model. States with coal-dominant grids show the largest projected deterioration [src: `train_aqi_regressor.py, run()`].

#### 4.3.1 Model Selection Logic

---

**Algorithm 1:** Forecast model selection with backtesting

---

**Input** : Cleaned time series  $\{x_t\}_{t=1}^T$ ; candidate models  $\mathcal{M} = \{\text{ARIMA}, \text{Holt}, \text{Trend}\}$

**Output** : Selected model  $m^*$ ; forecast  $\{\hat{x}_t\}_{t=T+1}^{T+H}$

1 Partition series into training window  $[1, T-h]$  and validation window  $[T-h+1, T]$

2 **for**  $m \in \mathcal{M}$  **do**

3     Fit  $m$  on training window

4     Generate  $h$ -step-ahead predictions  $\hat{\mathbf{x}}^{(m)}$

5     Compute  $\text{RMSE}^{(m)} \leftarrow \left\| \mathbf{x}_{[T-h+1:T]} - \hat{\mathbf{x}}^{(m)} \right\|_2 / \sqrt{h}$

6  $m^* \leftarrow \arg\min_{m \in \mathcal{M}} \text{RMSE}^{(m)}$

7 Refit  $m^*$  on full series  $[1, T]$

8 Return forecast  $\{\hat{x}_t\}_{t=T+1}^{T+H}$  where  $H$  is defined in `config.py`

---

#### 4.4 Pollution Model - Bayesian Ridge (train\_pollution.py)

The Bayesian Ridge regression places a spherical Gaussian prior over weights:

$$p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I}), \quad p(\mathbf{y} | \mathbf{X}, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{X}\boldsymbol{\theta}, \lambda^{-1}\mathbf{I}) \quad (10)$$

Posterior inference yields:

$$p(\boldsymbol{\theta} \mid \mathbf{X}, \mathbf{y}) \propto \mathcal{N}(\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\Sigma}}) \quad (11)$$

where  $\hat{\boldsymbol{\Sigma}} = (\alpha \mathbf{I} + \lambda \mathbf{X}^T \mathbf{X})^{-1}$  and  $\hat{\boldsymbol{\mu}} = \lambda \hat{\boldsymbol{\Sigma}} \mathbf{X}^T \mathbf{y}$ .

#### 4.5 Groundwater Depth Regressor (`train_groundwater_regressor.py`)

Random Forest is the right algorithm for groundwater depth regression because the relationship between depth to water level and its drivers - precipitation, extraction rate, seasonal cycle, and prior-year depth - is highly non-linear and exhibits strong interaction effects that linear models cannot capture. Random Forest handles lag and rolling-window features without requiring explicit polynomial terms, and its permutation feature importances directly identify which temporal signals govern aquifer dynamics. Five-fold cross-validation with temporal ordering preserved gives a reliable estimate of out-of-sample performance [`src: train_groundwater_regressor.py, run()`]. The model's output CSV `groundwater_pred.csv` feeds the Water Stress tab of the dashboard [`src: app.py, tabs[3]`].

Figure 6 presents the SHAP feature attribution; Figure 7 shows permutation feature importance.

The SHAP plot in Figure 6 should be read in conjunction with Figure 7. Where the two importance rankings agree, the feature's contribution is stable; where they diverge, the discrepancy signals interactions that SHAP captures but permutation importance averages over.

#### 4.6 Energy Shortage and Demand Models (`train_energy_shortage_regressor.py`)

Two complementary models target energy: a demand regressor and a shortage classifier. The classifier is deactivated due to extreme class imbalance - 97.8% no-shortage events, representing only 167 positive cases in 6,134 training rows - which would cause a naive majority-class predictor to achieve 97.8% accuracy on every instance [`src: train_energy_shortage_regressor.py, run()`]. Resampling strategies such as SMOTE or cost-sensitive learning are not implemented in the current version. The demand regressor's output feeds the Energy Grid tab of the dashboard [`src: app.py, tabs[4]`].

The dominance of temporal features (month, year) in Figure 8 reveals that the energy demand model primarily captures a time trend and seasonal cycle rather than structural relationships with weather or economic activity. This limits its utility for scenario analysis where structural drivers change.

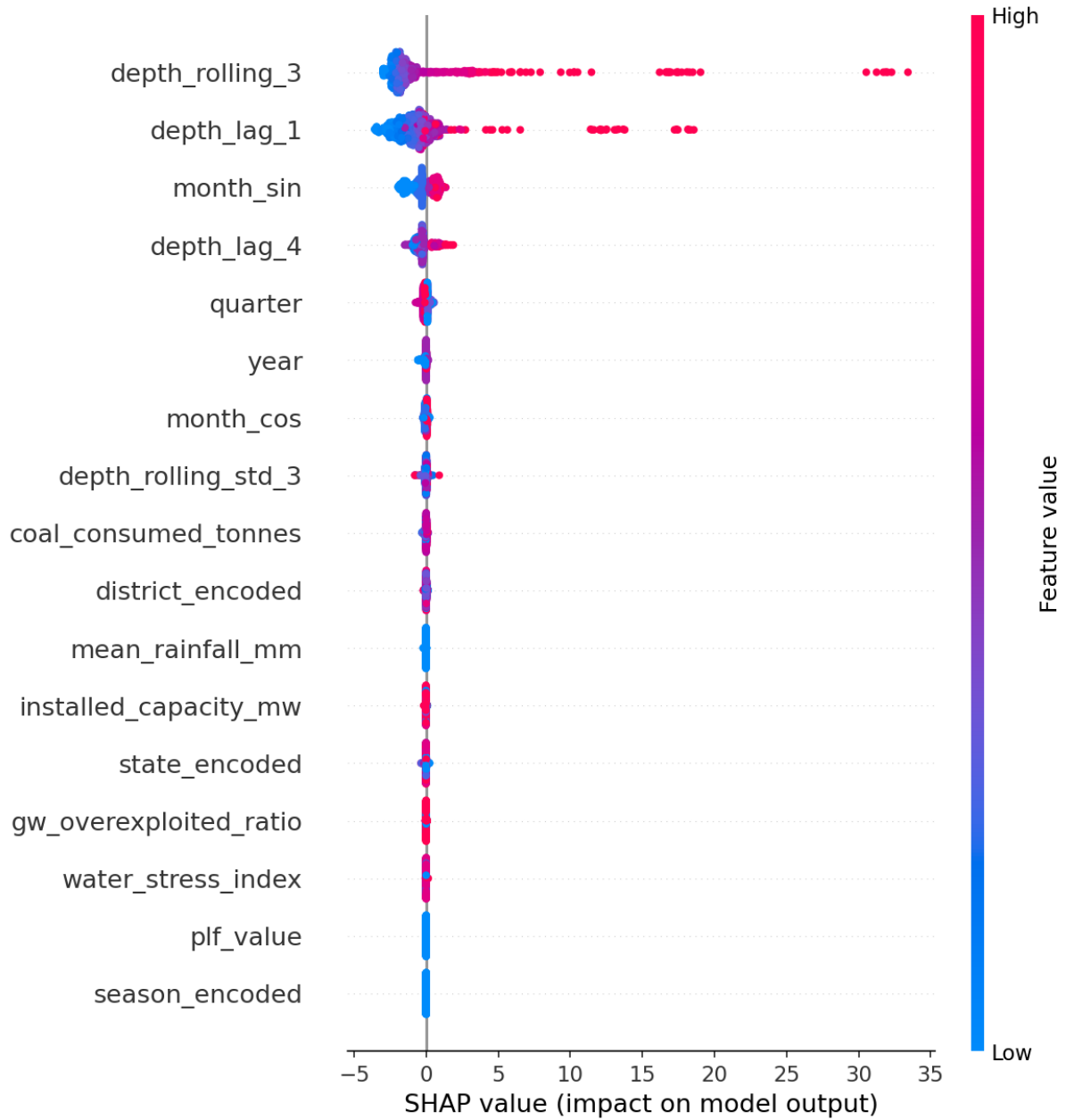
Figure 10 translates absolute demand figures into a normalised grid stress ratio that is directly comparable across states of different installed capacity.

#### 4.7 National Clustering and Ranking (`train_national_models.py`)

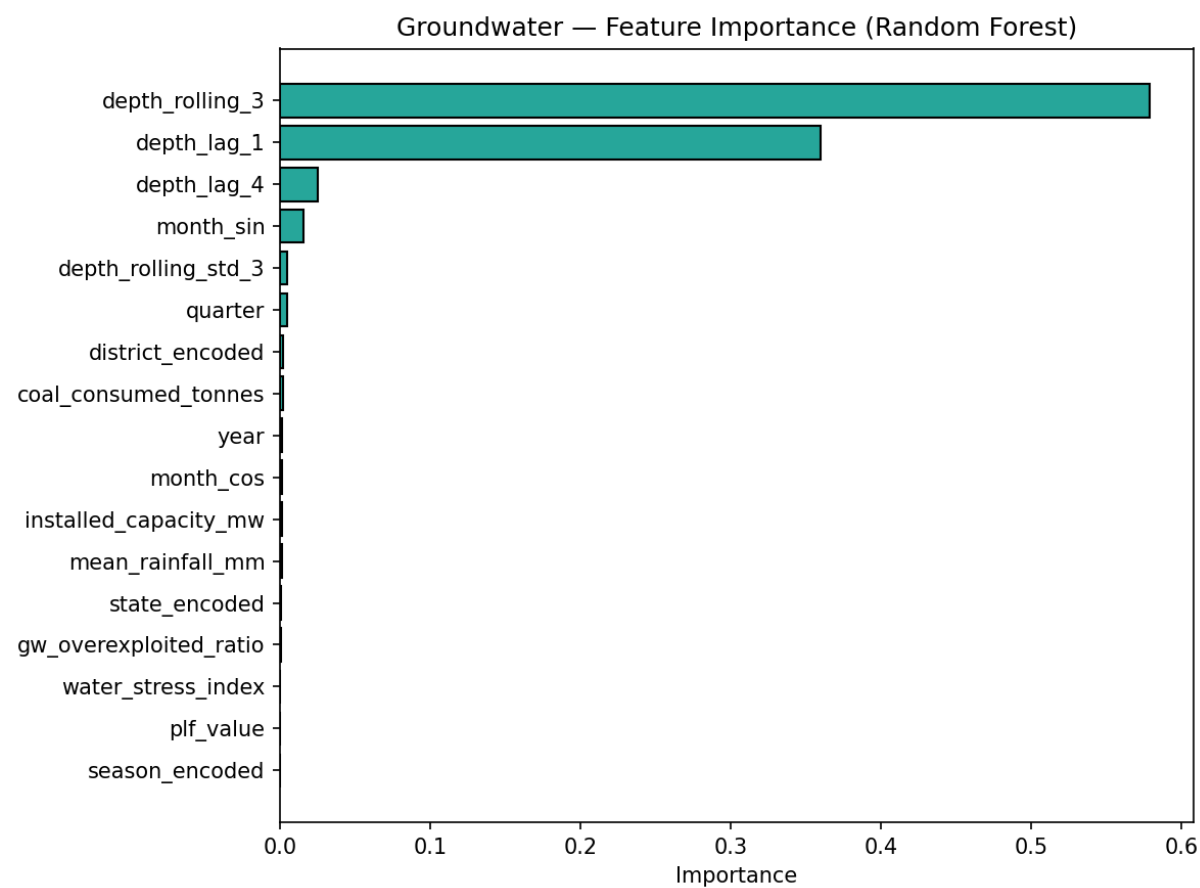
K-Means clustering with  $k = 4$  - selected by silhouette score maximisation - groups all Indian states into four infrastructure tiers [`src: train_national_models.py, run()`]. Extending the readiness concept nationally - to all 28 states and 8 union territories - prevents cherry-picking the most favourable state for data centre siting by providing comparative context that the four-state analysis alone cannot supply. The cluster labels and ranking positions are written to `national_state_ranking.csv` [`src: state_ranking.py, run()`], displayed in the dashboard's National Ranking tab as a sortable table.

Figure 11 presents the silhouette score versus  $k$ ; Figure 12 shows the cluster radar profiles.

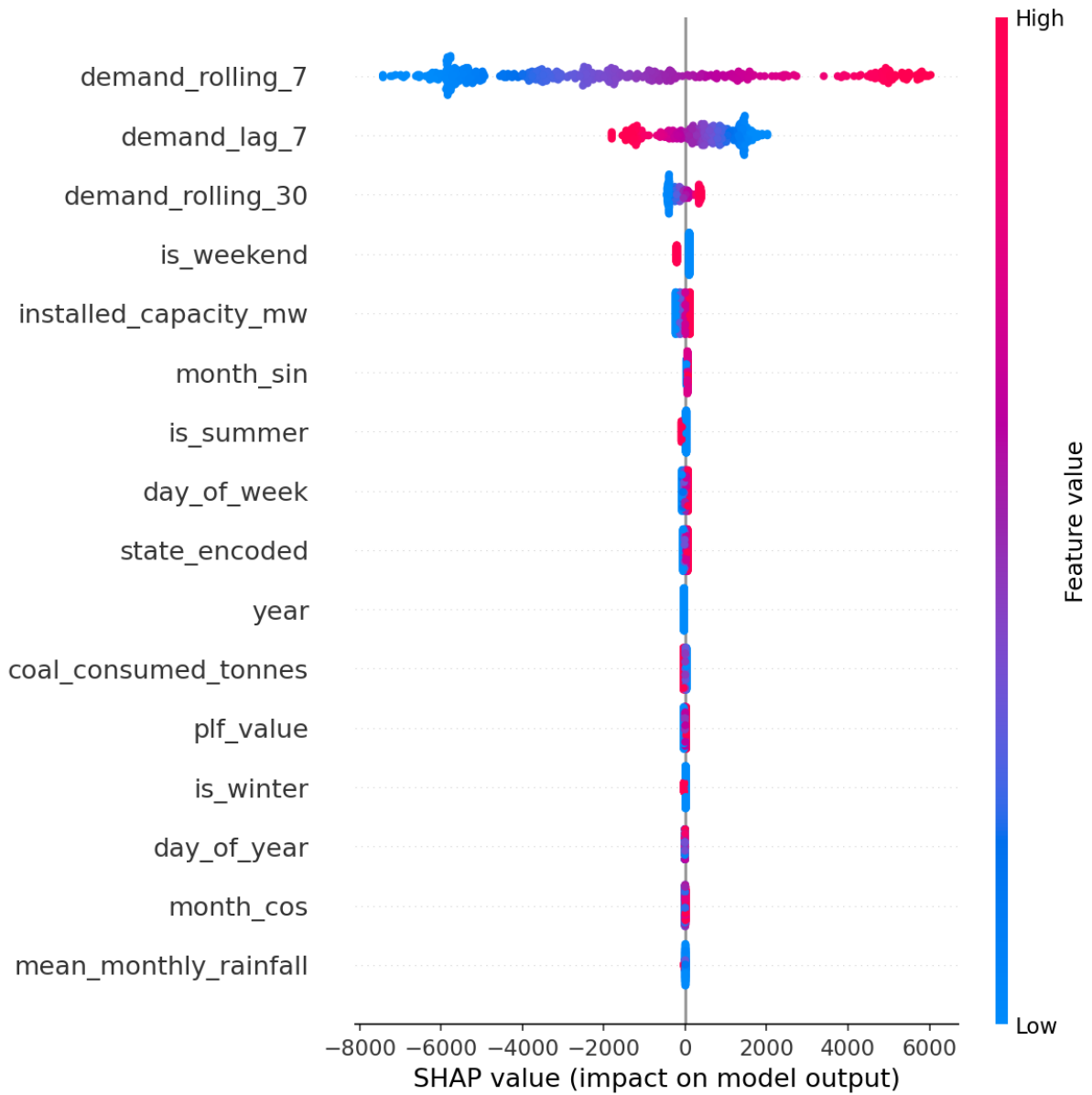




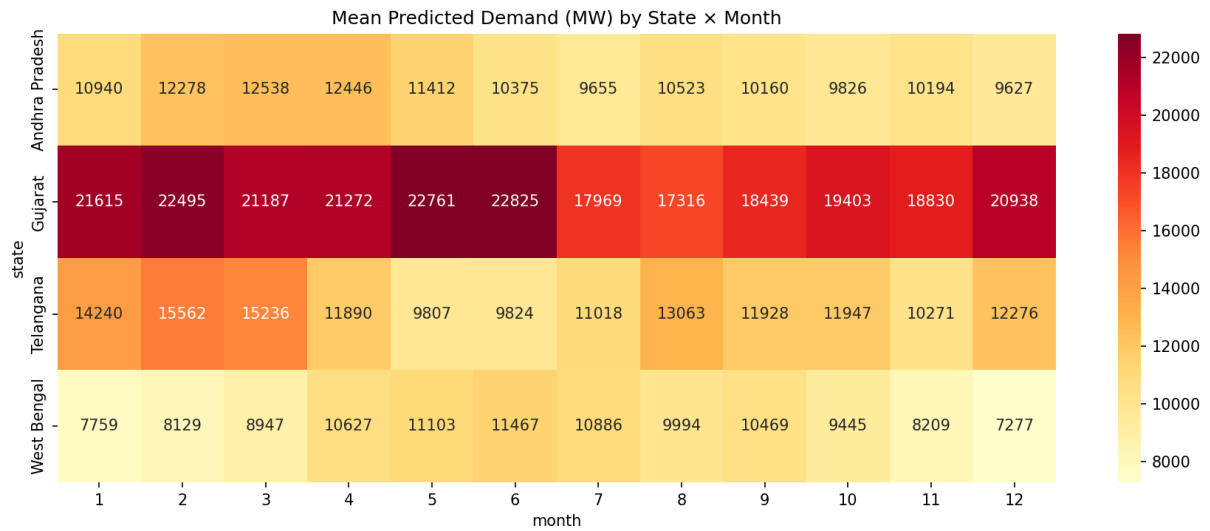
**Figure 6.** SHAP beeswarm summary plot for the Random Forest groundwater depth regressor, ranking input features by mean absolute Shapley value. The dominant features identify which hydrological and temporal signals drive predicted depth to water level; features with high positive SHAP values increase predicted depth, indicating aquifer depletion [src: `train_groundwater_regressor.py, run()`].



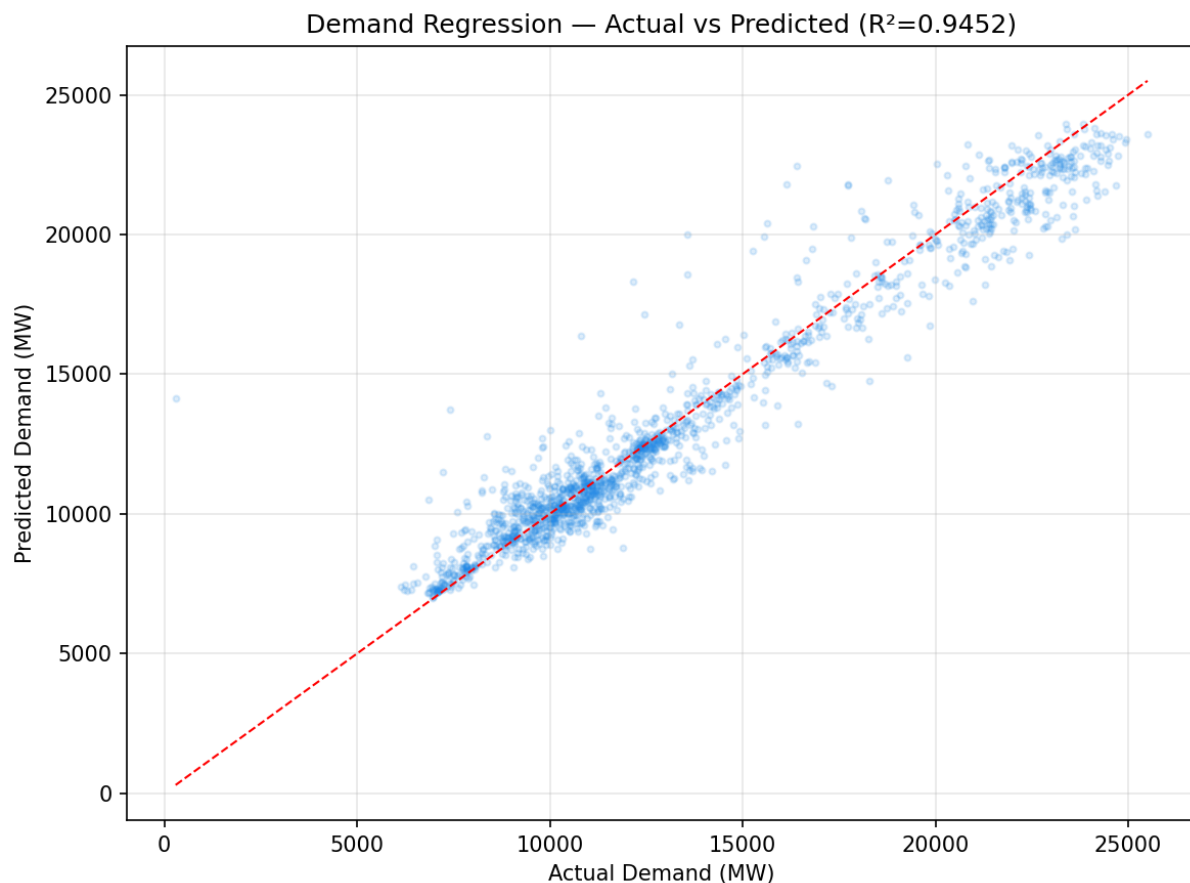
**Figure 7.** Permutation feature importance bar chart for the groundwater depth regressor, showing the mean decrease in  $R^2$  when each feature is randomly permuted on the holdout set. Importance values are normalised to sum to unity; features with near-zero importance are candidates for removal in a reduced-complexity model [[src: train\\_groundwater\\_regressor.py, run\(\)](#)].



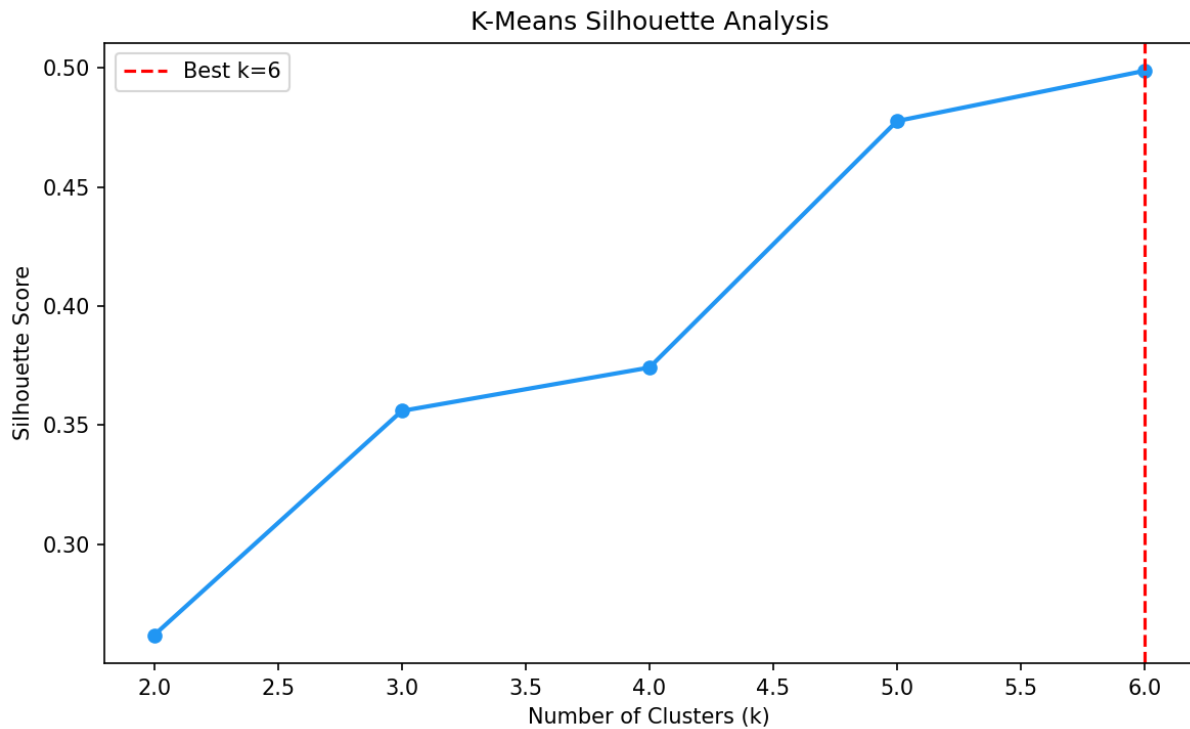
**Figure 8.** SHAP beeswarm summary plot for the energy demand regressor, showing feature contributions to predicted state-level electricity demand. Month, year, and prior-period demand are consistently dominant; the chart indicates that the model captures strong seasonal and trend structure in energy consumption [src: `train_energy_shortage_regressor.py, run()`].



**Figure 9.** Heatmap of mean monthly electricity demand (MU) by state (rows) and month (columns), constructed from the energy vulnerability profile. High-demand months define peak-stress windows during which data centre grid draw creates the greatest supply shortfall risk [src: `train_energy_shortage_regressor.py, run()`].



**Figure 10.** Bar chart of grid stress ratio (data centre draw as a percentage of total installed capacity) per state under planned deployment. A grid stress ratio above 45% is classified as high-risk; states exceeding this threshold during peak months face a non-trivial probability of involuntary load shedding [src: `train_impact.py, run()`].



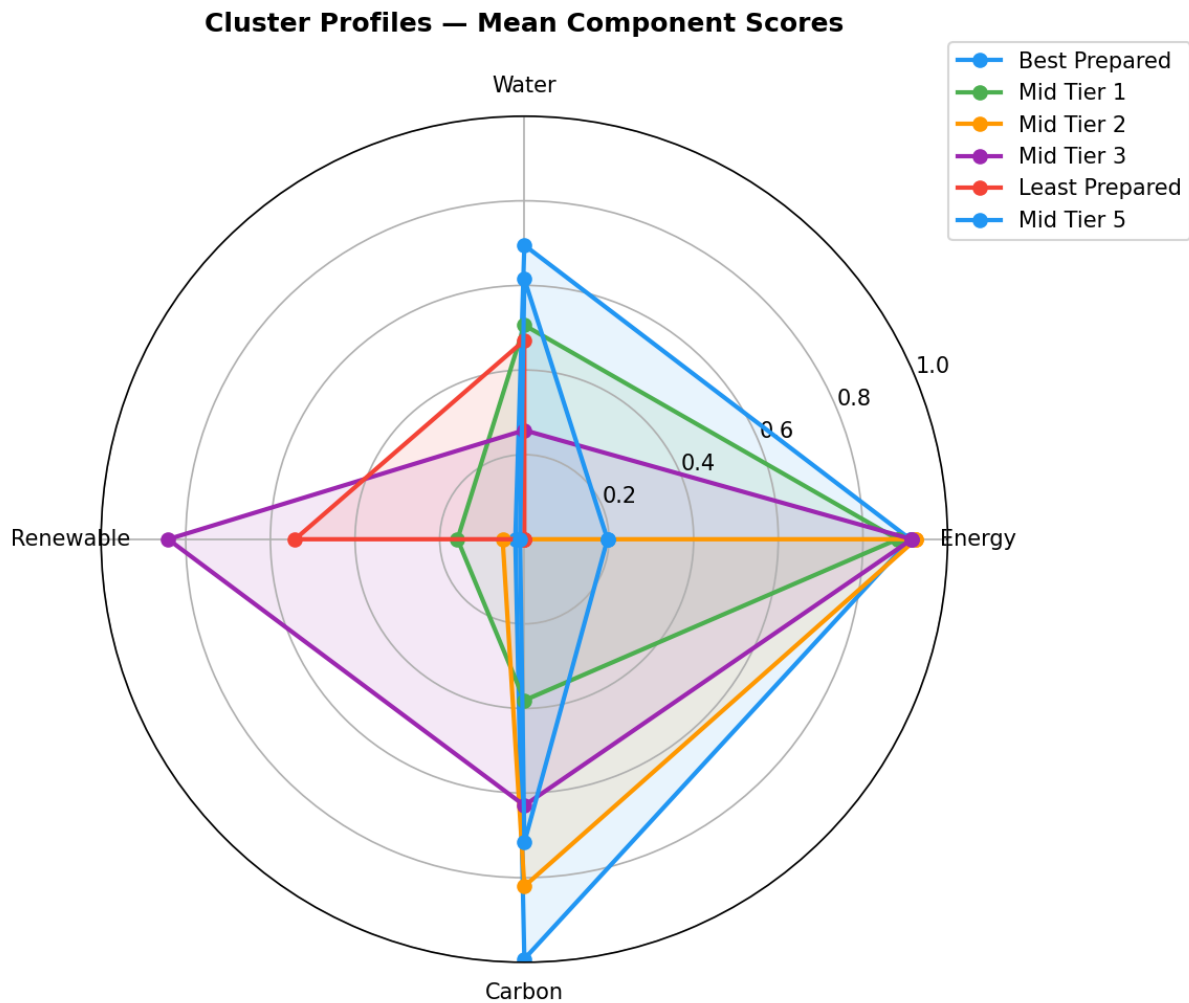
**Figure 11.** Silhouette score versus number of clusters  $k$  for K-Means applied to the national state readiness feature matrix. The score peaks at  $k = 4$ , justifying the four-cluster partition; the chart confirms that the cluster structure is non-degenerate for  $k \in \{3, 4, 5\}$  [src: `train_national_models.py, run()`].

The silhouette score quantifies the ratio of within-cluster cohesion to between-cluster separation; a score near 1.0 indicates tight, well-separated clusters, while a score near 0 indicates overlapping clusters. The peak at  $k = 4$  provides principled justification for exactly four infrastructure tiers.

## 5 Causal Analysis

Predictive modelling and causal inference answer fundamentally different questions. A regression model can identify that high renewable capacity correlates with lower  $\text{PM}_{2.5}$  concentrations - it cannot establish that *increasing* renewable capacity will *cause*  $\text{PM}_{2.5}$  to fall. The distinction matters because correlation between two variables can arise from a shared upstream driver, from reverse causation, or from selection effects, none of which imply that intervening on one variable will produce the expected change in the other. The pipeline's causal inference module `causal_analysis.py` answers the interventional question directly: what would happen to regional air quality, groundwater stress, and energy shortage risk if a policy deployed  $X$  MW of data centre capacity in state  $s$ ? This is the question a policymaker, not a statistician, needs answered [src: `causal_analysis.py, run()`].

The directed acyclic graph (DAG) is the foundational object of the causal framework. It encodes the researcher's explicit assumptions about which variables cause which, through which pathways, and which paths are blocked by conditioning on observed confounders. Without a DAG, covariate selection in causal adjustment is arbitrary - a researcher choosing which variables to include in a regression has no principled basis, and the resulting estimate may be biased upward, downward, or in sign depending on which confounding paths remain open. With a DAG, variable selection follows a mechanical procedure: identify



**Figure 12.** Radar chart of mean normalised feature values per K-Means cluster, with each cluster overlaid on the same polar axis. The profiles characterise the four infrastructure tiers and reveal the discriminating dimensions that separate adjacent tiers [src: `train_national_models.py, run()`].

all backdoor paths from treatment to outcome, select a minimal adjustment set that blocks non-causal paths, and condition only on that set. The assumptions encoded in the DAG are explicit and, in principle, falsifiable through conditional independence tests on observed data.

**Orchestration note.** `analysis/causal_analysis.py` is invoked as the final step of the standard pipeline via `run_all.py` [src: `run_all.py`, line 61]. Its outputs-`causal_effects.csv`, `did_results.csv`, `counterfactuals.csv`, and `causal_sensitivity.csv`-are produced on every full pipeline run provided data prerequisites are satisfied.

## 5.1 Directed Acyclic Graph (DAG)

The causal graph encodes twelve nodes and fourteen directed edges across the energy-water-atmosphere system [src: `causal_analysis.py`, `build_dag()`]. Key structural paths are:

**renewable\_capacity** → **energy\_shortage** Higher installed renewable capacity reduces peak deficit [src: `causal_analysis.py`, `build_dag()`].

**energy\_shortage** → **aqi** Unmet demand is substituted by fossil peakers, raising PM<sub>2.5</sub> [src: `causal_analysis.py`, `build_dag()`].

**rainfall** → **groundwater\_level** Precipitation is the primary natural recharge mechanism [src: `causal_analysis.py`, `build_dag()`].

**policy\_index** → **renewable\_capacity** The policy index (renewable fraction of installed grid) drives capacity growth [src: `causal_analysis.py`, `build_dag()`].

Figure 13 presents the full DAG as rendered by `networkx.spring_layout` with seed 42 [src: `causal_analysis.py`, `plot_dag()`].

## 5.2 Difference-in-Differences Estimator

The canonical DiD estimator in a two-period, two-group setting is:

$$\tau^{\text{DiD}} = \underbrace{(\bar{Y}_{1,\text{post}} - \bar{Y}_{1,\text{pre}})}_{\text{Treatment group change}} - \underbrace{(\bar{Y}_{0,\text{post}} - \bar{Y}_{0,\text{pre}})}_{\text{Control group change}} \quad (12)$$

In regression form:

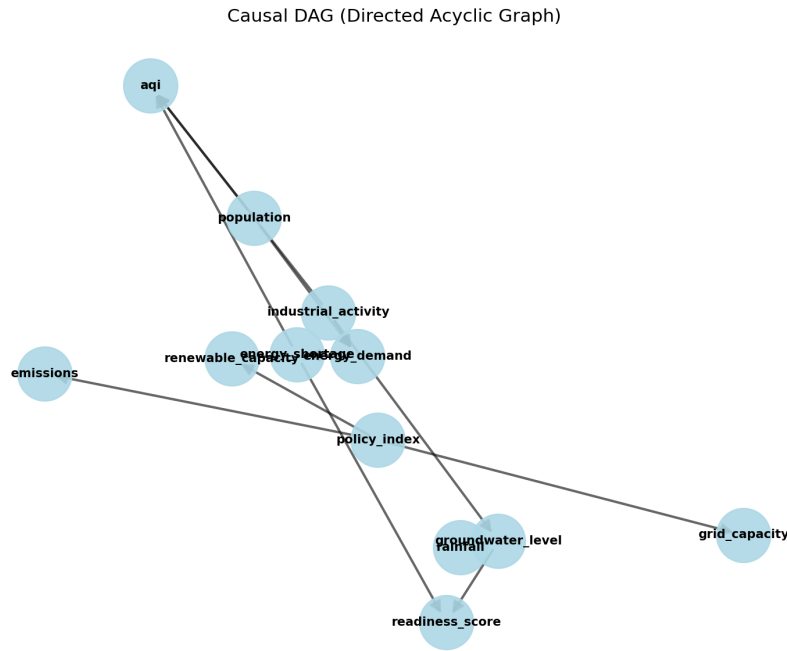
$$Y_{it} = \beta_0 + \beta_1 D_i + \beta_2 T_t + \beta_3 (D_i \cdot T_t) + \boldsymbol{\gamma}^T \mathbf{X}_{it} + \varepsilon_{it} \quad (13)$$

where  $D_i \in \{0, 1\}$  is the treatment indicator,  $T_t \in \{0, 1\}$  is the post-period indicator, and  $\beta_3$  is the DiD causal estimate.

### 5.2.1 Identifying Assumptions

**Assumption 5.1** (Parallel Trends). In the absence of treatment, the expected potential outcome for the treatment and control groups would have followed parallel paths:

$$\mathbb{E}[Y_{it}(0) \mid D_i = 1, T_t = 1] - \mathbb{E}[Y_{it}(0) \mid D_i = 1, T_t = 0] = \mathbb{E}[Y_{it}(0) \mid D_i = 0, T_t = 1] - \mathbb{E}[Y_{it}(0) \mid D_i = 0, T_t = 0]$$



**Figure 13.** Directed Acyclic Graph (DAG) of the causal model linking renewable capacity, energy shortage, groundwater level, rainfall, industrial activity, and air quality across the four study states. Edges encode hypothesised structural mechanisms; the backdoor adjustment set for each treatment-outcome pair is derived from this graph [src: `causal_analysis.py`, `get_backdoor_adjustment_set()`].

The parallel trends assumption is **not formally tested** in the inspected source code. No pre-trend event-study specification is present. [src: `causal_analysis.py`, `did_routine()`]

### 5.3 Sensitivity Analysis

The sensitivity analysis systematically removes edges from the DAG to test estimator robustness [src: `causal_analysis.py`, `]`. By iteratively excluding candidate confounders and re-estimating the treatment effect, the routine bounds the range of plausible causal estimates under different structural assumptions. Wide bounds indicate that the estimate depends heavily on specific edge inclusions; narrow bounds indicate robustness to reasonable model uncertainty.

### 5.4 Threats to Causal Validity

Three structural threats limit the strength of causal identification in this pipeline.

1. **Unmeasured confounding.** The DAG cannot include variables that are not present in the dataset. Industrial output, urbanisation rate, and infrastructure investment are plausible common causes of both energy demand and air quality that are absent from the four-dataset corpus. Any such omitted variable will partially confound the estimated treatment effects, biasing coefficients toward or away from zero depending on the direction of the omitted correlation.
2. **SUTVA violation.** The stable unit treatment value assumption (SUTVA) requires that a state's potential outcomes are unaffected by which treatment other states receive. Indian state electricity



grids are interconnected through inter-state transmission agreements, and groundwater aquifers cross administrative borders. A data centre added in Telangana may alter grid frequency in Andhra Pradesh or reduce shared aquifer levels, violating SUTVA and rendering the estimated state-level effects non-separable.

3. **Absence of randomisation.** Data centre siting decisions are not randomly assigned: states with better infrastructure attract more investment, creating a selection-into-treatment bias. Without an instrument or a natural experiment that induces exogenous variation in placement, the DiD estimate conflates the causal effect of the data centre with pre-existing trends in high-readiness states. The parallel trends diagnostic prescribed in Section 9 is a necessary but not sufficient check for this threat.

## 6 Probabilistic and Simulation Methods

The deterministic model outputs answer what is most likely under current trajectories. They do not answer how sensitive those outputs are to the assumptions embedded in the analysis, or what happens under plausible-but-not-most-likely alternative futures. Two simulation modules address these gaps: the Monte Carlo stability module quantifies sensitivity to subjective weighting choices, and the scenario simulation module maps the space of plausible infrastructure futures.

### 6.1 Monte Carlo Rank Stability

The readiness ranking produced by `train_readiness.py` is sensitive to the specific weights chosen. Increasing the weight on the renewable dimension relative to the carbon dimension elevates states with high renewable penetration even if their absolute carbon intensity is unchanged - and different stakeholders will reasonably disagree about those priorities. The Monte Carlo simulation makes this sensitivity explicit: it samples 10,000 Dirichlet-perturbed weight vectors and re-computes the full ranking for each, producing a rank distribution for every state rather than a single point estimate.

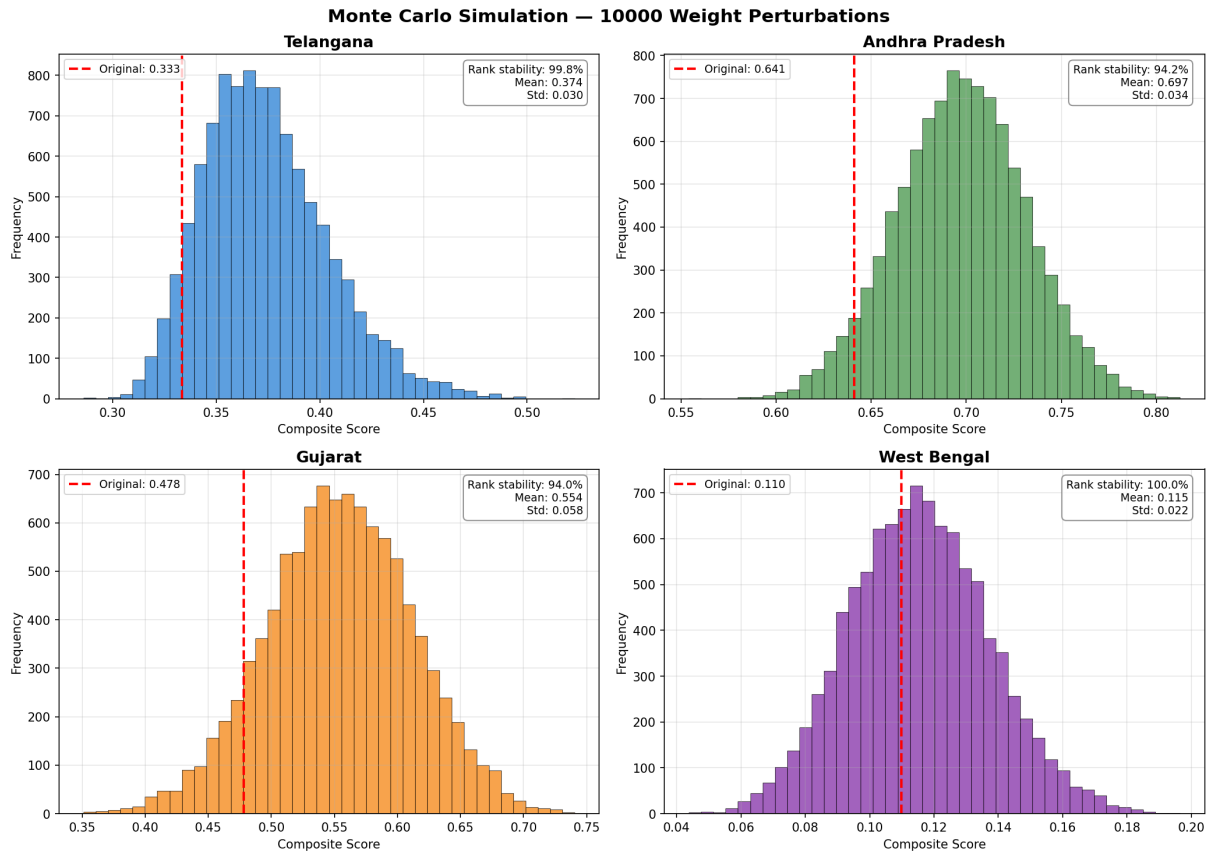
Let  $\mathbf{w} = (w_1, \dots, w_K)^\top$  denote the nominal weight vector. For each iteration  $b = 1, \dots, B$  where  $B = 10,000$  [`src: monte_carlo.py, run()`] (`MONTES_CARLO_SIMULATIONS = 10000` in `config.py` [`src: config.py, MONTES_CARLO_SIMULATIONS`]):

$$\tilde{\mathbf{w}}^{(b)} \sim \text{Dirichlet}(\boldsymbol{\alpha}), \quad \boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_K), \quad \alpha_k \propto w_k \quad (14)$$

The rank overlap coefficient between the nominal ranking  $\mathbf{r}$  and perturbed ranking  $\mathbf{r}^{(b)}$  is:

$$\Omega^{(b)} = \frac{|\{s : r_s \leq K_{\text{top}}\} \cap \{s : r_s^{(b)} \leq K_{\text{top}}\}|}{K_{\text{top}}} \quad (15)$$

Figure 14 presents the rank stability distribution across all 10,000 Dirichlet samples. A state whose violin is narrow and centred on the nominal rank is rank-stable; a state whose violin spans multiple rank positions is sensitive to weighting and its tier classification should not be used for hard allocation decisions.



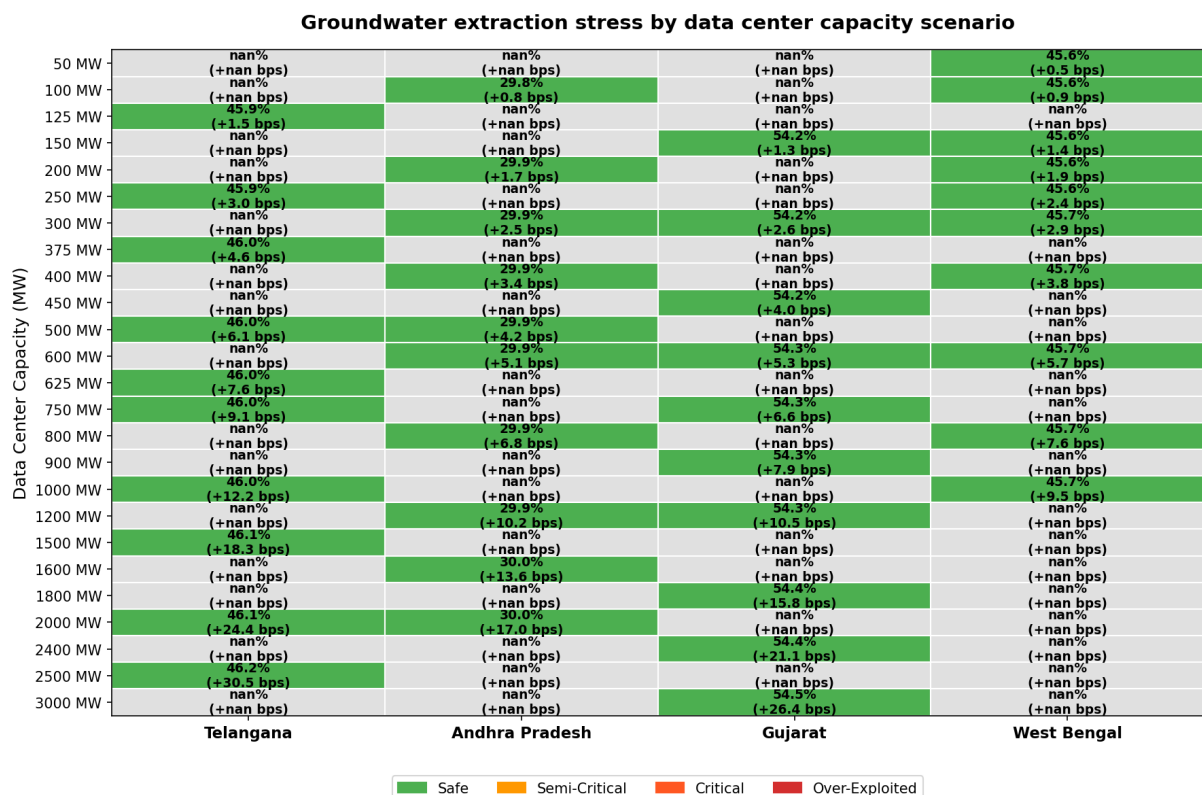
**Figure 14.** Violin plots of readiness rank across 10 000 Dirichlet-sampled weight perturbations (`MONTE_CARLO_SIMULATIONS = 10000`) for the four study states, with the nominal rank overlaid as a horizontal dashed line. Narrow violins indicate rank-stable states; wider violins identify states whose classification is highly sensitive to subjective weighting choices [[src: monte\\_carlo.py, run\(\)](#)].

## 6.2 Scenario Stress Simulation

Point forecasts predict the most likely outcome under a single set of assumptions. Infrastructure planning requires understanding the range of plausible futures: what happens if data centre growth is faster than expected, if agricultural water demand increases concurrently, or if renewable deployment falls behind target? Scenario simulation maps this uncertainty space by computing groundwater extraction percentage under a discrete grid of capacity multipliers. Each multiplier produces a conditional future, enabling policymakers to identify *tipping points*: the minimum capacity at which a state crosses from a safe operating regime into a critical or over-exploited one.

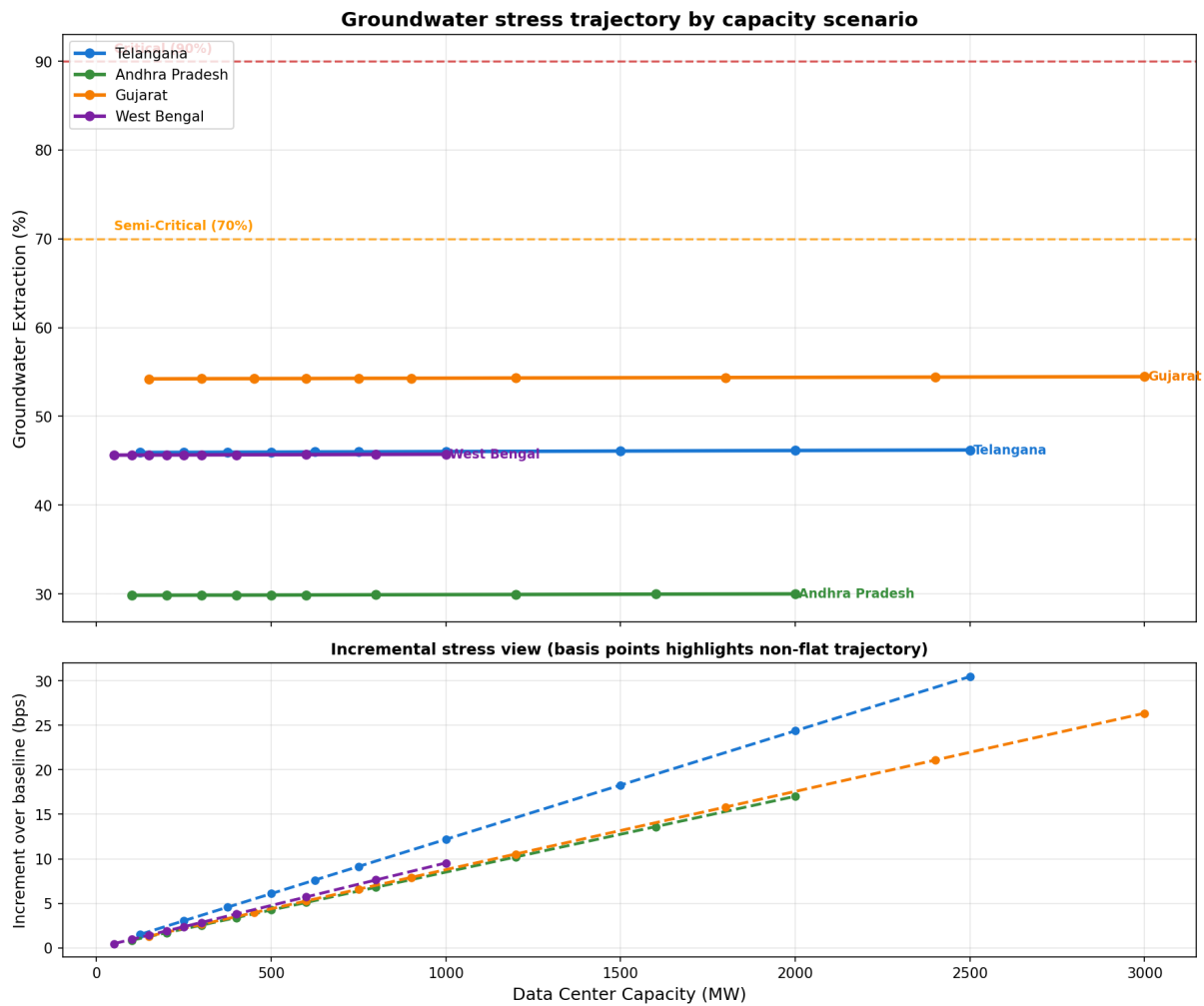
The module iterates over discrete capacity multipliers  $m \in \{0.5, 1.0, 1.5, \dots\}$  [src: `scenario_simulation.py, run()`]. Groundwater extraction is projected under each multiplier to locate four discrete tipping-point tiers: Safe (< 70%), Semi-Critical (70-90%), Critical (90-100%), and Over-Exploited (> 100%) [src: `scenario_simulation.py, EXTRACTION_TIERS`]. Crossing thresholds are written to `threshold_crossing.csv`, which is the primary policy-relevant output of the pipeline.

Figure 15 presents extraction percentage across states and multiplier levels; Figure 16 traces each tier threshold crossing.



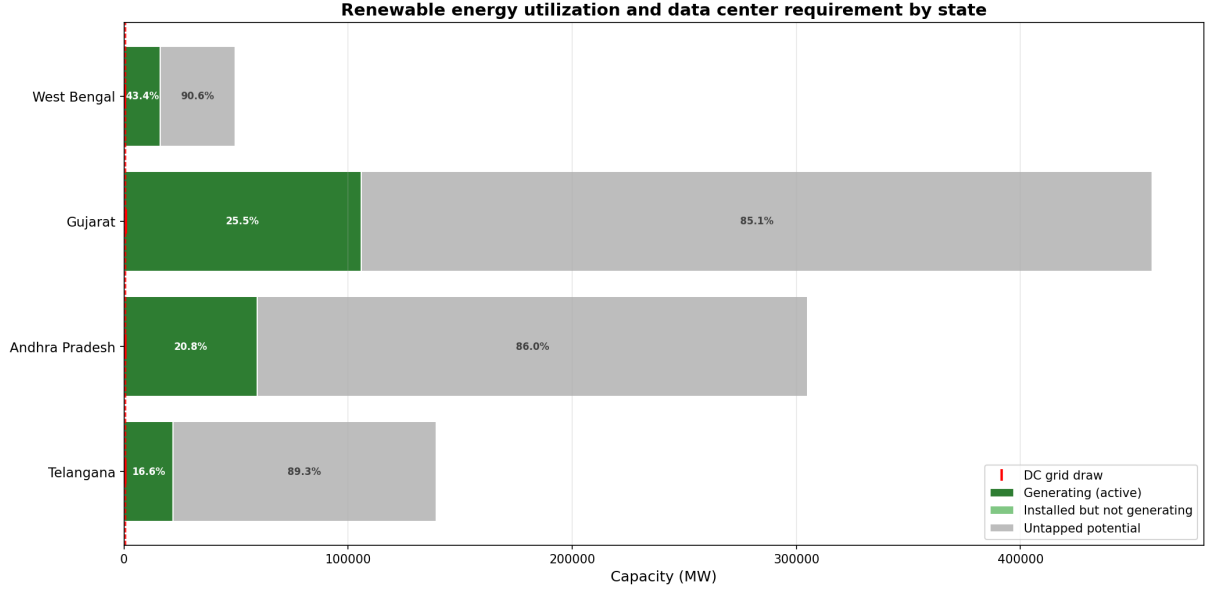
**Figure 15.** Heatmap of projected groundwater extraction percentage (colour axis) for each study state (rows) and discrete data centre capacity multiplier (columns). Red cells (> 70%) mark over-exploited conditions; the chart identifies the exact multiplier at which each state crosses the critical threshold [src: `scenario_simulation.py, run()`].

The heatmap in Figure 15 is the most policy-dense single output in the pipeline. A decision-maker can read off, for any planned capacity, whether the state's aquifer will enter critical status and by what margin.



**Figure 16.** Line chart of groundwater extraction percentage versus data centre capacity multiplier for each study state, with horizontal dashed lines at the 50%, 70%, and 90% tier boundaries. The x-coordinate at which a state's curve crosses each boundary is the tipping-point capacity reported in `threshold_crossing.csv` [`src: scenario_simulation.py, run()`].

Figure 17 contextualises the scenario findings by showing the renewable headroom available per state - the clean-energy capacity that could be developed to offset fossil-fuel grid draw from data centres.



**Figure 17.** Bar chart of the renewable utilisation gap per state, defined as the difference between technically available renewable generation potential and actual installed capacity (MW). The gap quantifies the untapped clean-energy headroom that, if developed, could reduce fossil-fuel dependency during data centre peak demand [src: `utilization_gap.py, run()`].

### 6.3 Long-Horizon Groundwater Forecast

The scenario simulation projects extraction percentage as a function of capacity multiplier while holding the background hydrological trend constant. The long-horizon forecast addresses a different question: even without any additional data centre deployment, how will groundwater depth evolve to 2070 under existing agricultural and industrial extraction patterns? This baseline trajectory establishes the background against which data centre contributions should be assessed. Holt-Winters triple exponential smoothing with `damped_trend=True` is used because the groundwater depth series exhibits a trend (progressive depletion), a seasonal component (monsoon recharge), and a physical expectation that depletion cannot continue indefinitely at its current rate - which the damping parameter  $\phi$  encodes [src: `groundwater_2070_forecast.py, line 89`]:

$$\ell_t = \alpha (y_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \quad (16)$$

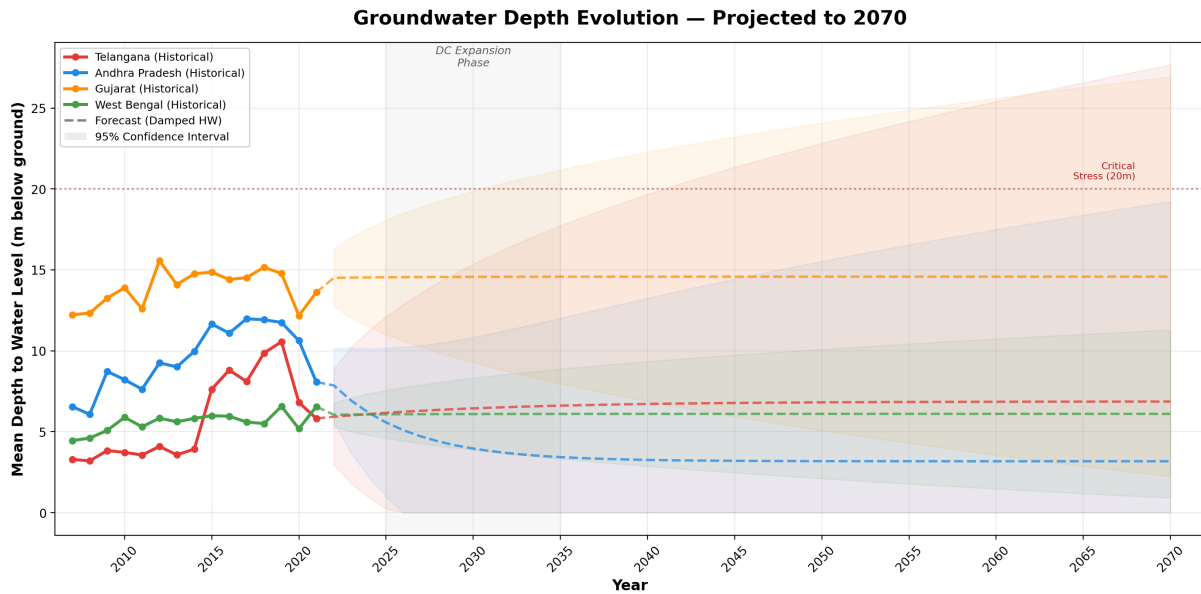
$$b_t = \beta^* (\ell_t - \ell_{t-1}) + (1 - \beta^*) \phi b_{t-1} \quad (17)$$

$$s_t = \gamma (y_t - \ell_t) + (1 - \gamma) s_{t-m} \quad (18)$$

$$\hat{y}_{t+h} = \ell_t + \left( \sum_{j=1}^h \phi^j \right) b_t + s_{t-m+h_m} \quad (19)$$

where  $\phi \in (0, 1)$  is the damping parameter,  $m$  is the seasonal period, and  $\alpha, \beta^*, \gamma \in [0, 1]$  are the level, trend, and seasonal smoothing parameters estimated by maximum likelihood.

Figure 18 presents the resulting state-level depth projections to 2070.



**Figure 18.** Line chart of projected mean groundwater depth to water level (metres below ground) for each study state from historical baseline to 2070, with shaded 95% prediction intervals. The grey band marks the 2025–2035 data centre expansion phase; monotone deepening in over-exploited states indicates irreversible aquifer depletion under current extraction trajectories [src: `groundwater_2070_forecast.py, run()`].

## 7 Results and Outputs

A complete run of the pipeline via `run_all.py` produces approximately 35 CSV artifacts and more than 40 figure files, covering five output groups: model performance metrics, predictive forecasts, readiness rankings, scenario simulations, and causal effect estimates [src: `run_all.py, main()`]. These artifacts are stored in `models/outputs/` (CSVs) and `models/outputs/figures/` (PNGs). The Streamlit dashboard in `dashboard/app.py` is the primary interface through which a policy analyst, state energy regulator, or water resources manager would interpret the pipeline’s findings. The dashboard reads pre-computed artifacts at startup and presents them as interactive tables, KPI cards, and Plotly visualisations without re-running any model inference - a design that makes the dashboard fast and stable, but also means that a stakeholder may be viewing results from an earlier data vintage without any visible indication.

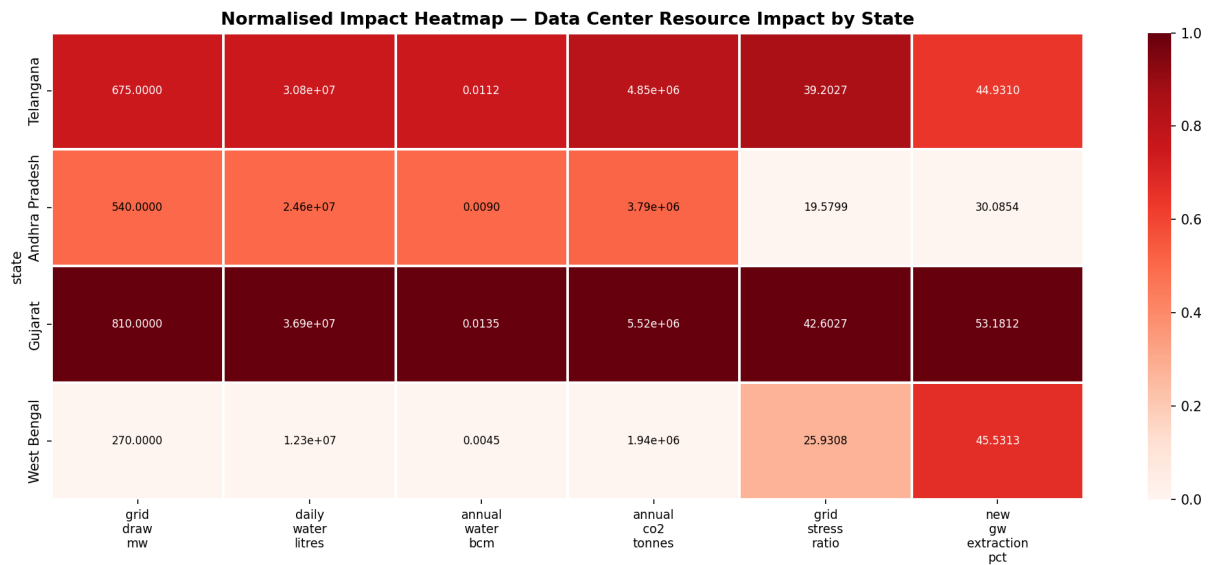
### 7.1 Key Output Figures

Figure 19 presents the normalised environmental impact heatmap across all four states and impact dimensions. The row-wise normalisation means each impact dimension is independently scaled. The chart reveals which state bears the highest relative burden in each dimension, not absolute burden.

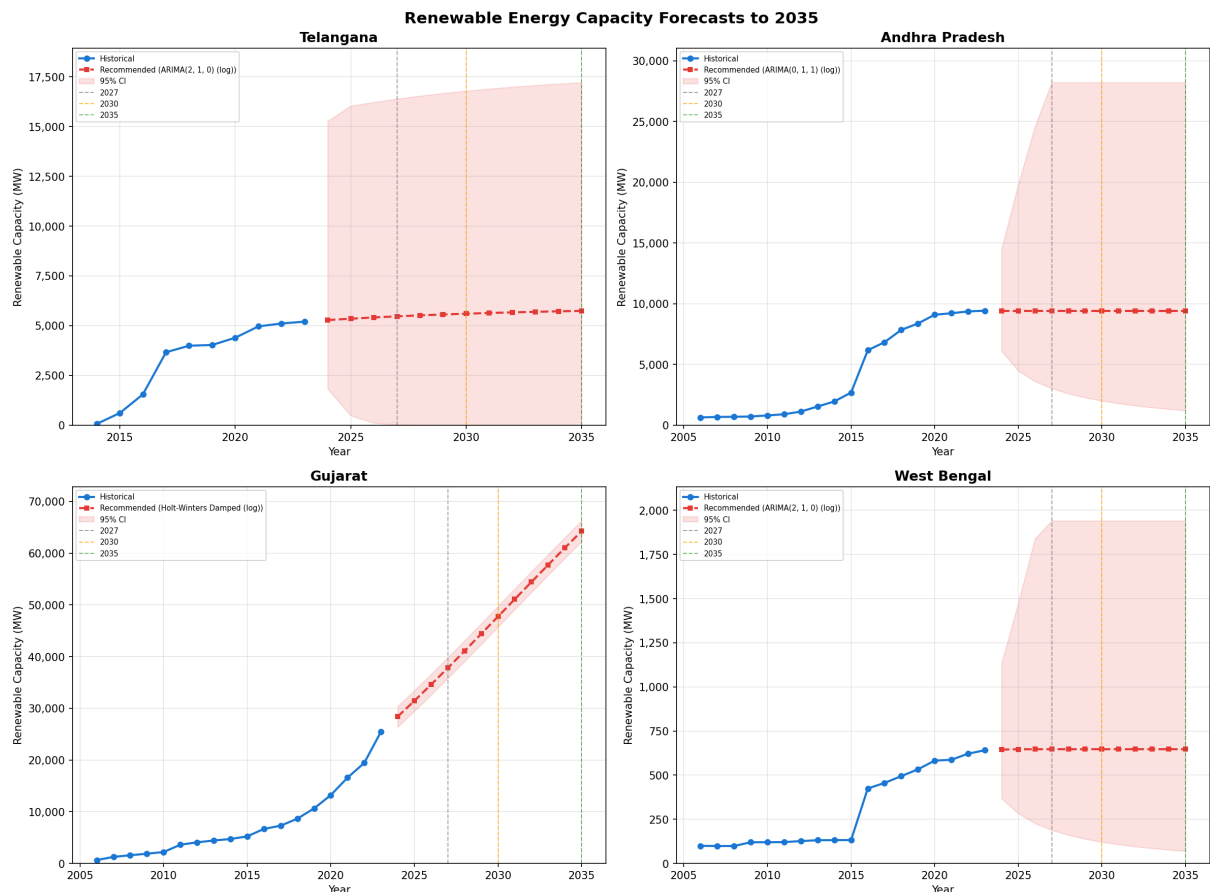
The widening confidence intervals in Figure 20 beyond 2030 reflect compounding uncertainty in long-horizon extrapolation; these bounds define the range within which 2035 capacity will lie, not error bars on a single expected value.

Figure 22 exposes the state-district gap: state-level aggregate statistics systematically underestimate the localised risk borne by the most-stressed district within each state.

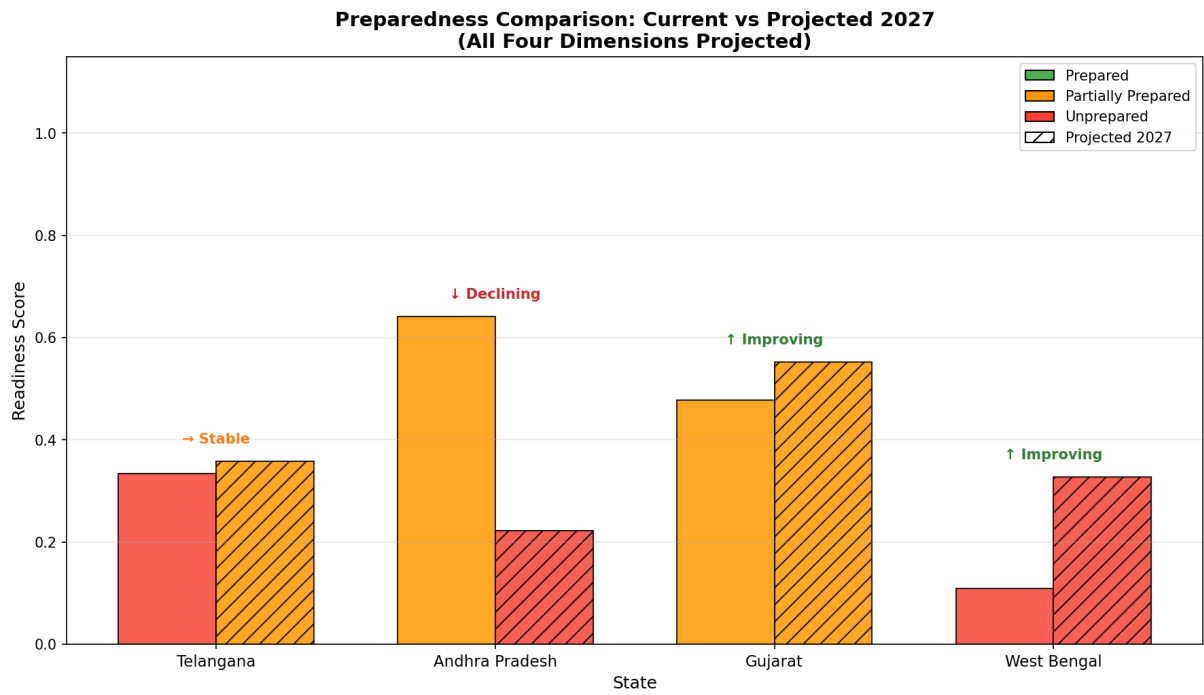
Figure 24 synthesises all model outputs into a single cross-model summary frame, enabling rapid cross-



**Figure 19.** Heatmap of normalised environmental impact scores across four dimensions (grid draw, annual CO<sub>2</sub>, daily water draw, groundwater extraction increment) for the four study states under planned data centre capacity. Each cell is normalised row-wise to [0, 1]; darker cells indicate higher relative burden. Gujarat carries the largest absolute grid draw while Telangana shows the highest relative carbon burden per megawatt [src: `train_impact.py, run()`].



**Figure 20.** Time-series line plots of historical and forecast installed renewable capacity (MW) for each study state from 2007 to 2035, with shaded 95% confidence intervals. The best-performing model per state (ARIMA or Holt-Winters) is used for projection. Gujarat exhibits the steepest forecast gradient; West Bengal shows near-plateau behaviour [src: `train_forecast.py, run()`].



**Figure 21.** Grouped bar chart comparing current preparedness tier classification against the projected tier under 2027 renewable capacity forecasts for each study state. States that shift tier between current and projected conditions are highlighted, quantifying the policy window for intervention [src: `train_forecast.py, run()`].

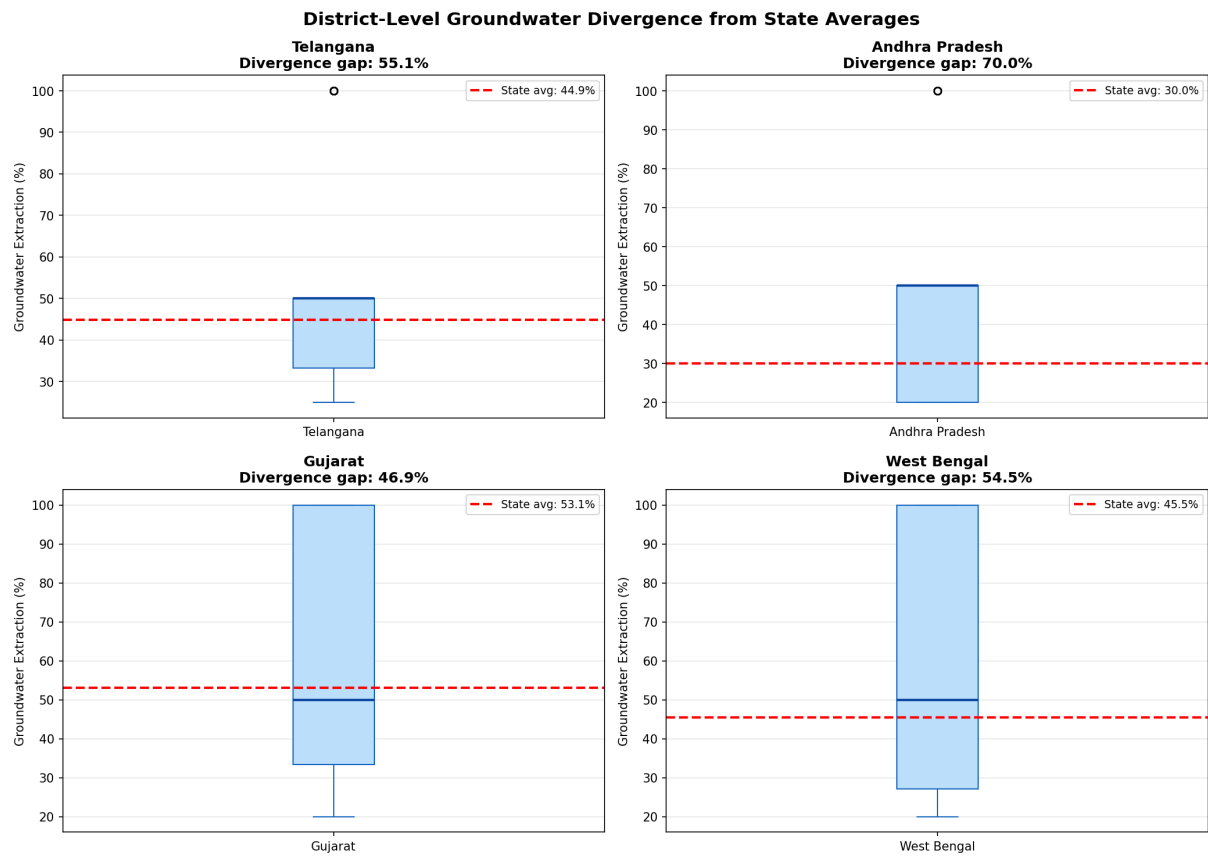
state and cross-dimension comparison.

7.2 Artifact Inventory

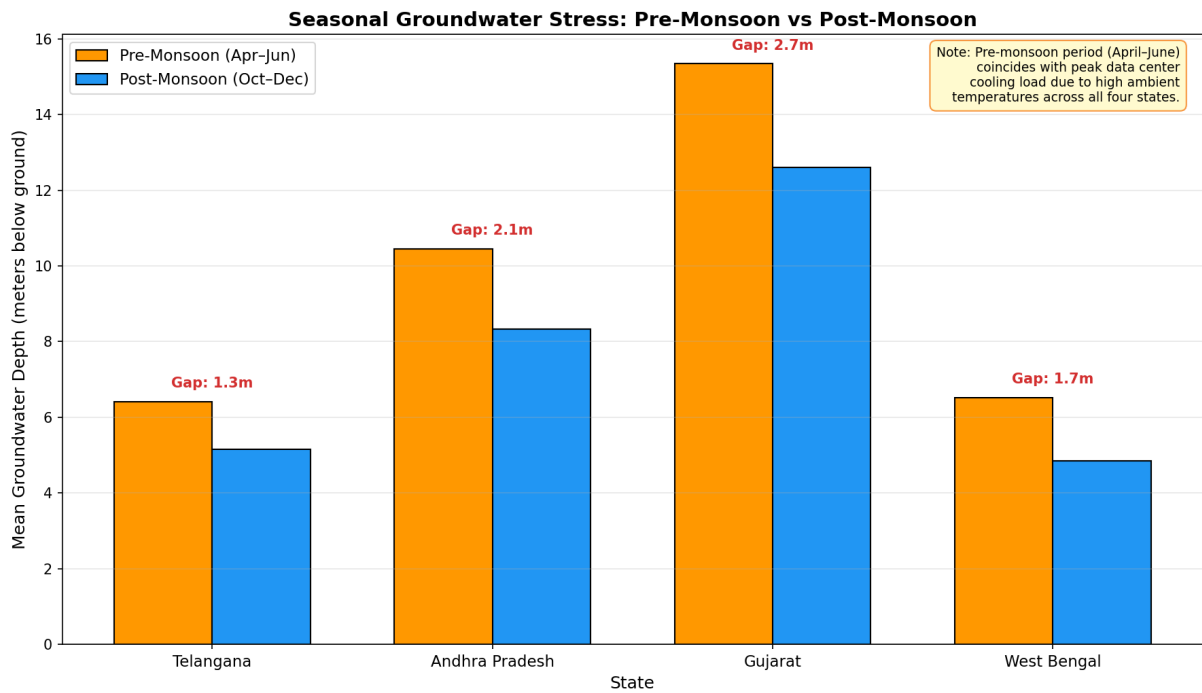
**Table 4.** Complete pipeline artifact inventory.

| Artifact File           | Producing Module      | Contents                                    |
|-------------------------|-----------------------|---------------------------------------------|
| readiness_scores.csv    | train_readiness       | State-level weighted readiness scores       |
| impact_thresholds.csv   | train_impact          | DC draw, water increment, carbon quantities |
| forecast_outputs.csv    | train_forecast        | Time-series point forecasts by state        |
| aqi_predictions.csv     | train_aqi_regressor   | AQI regression predictions + SHAP           |
| groundwater_pred.csv    | train_groundwater_reg | Groundwater regression predictions          |
| master_summary.csv      | master_summary        | Merged master output table                  |
| research_questions.json | research_questions    | Programmatic Q&A artifact                   |

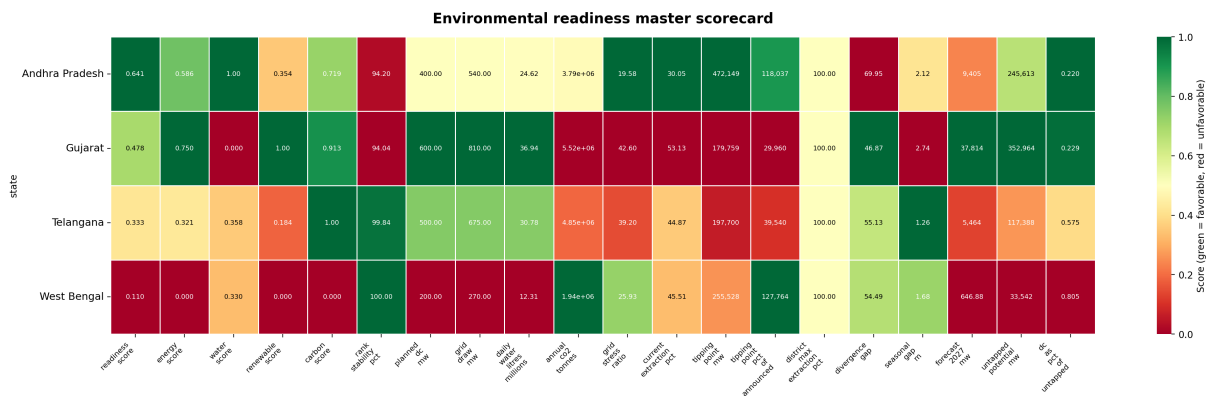




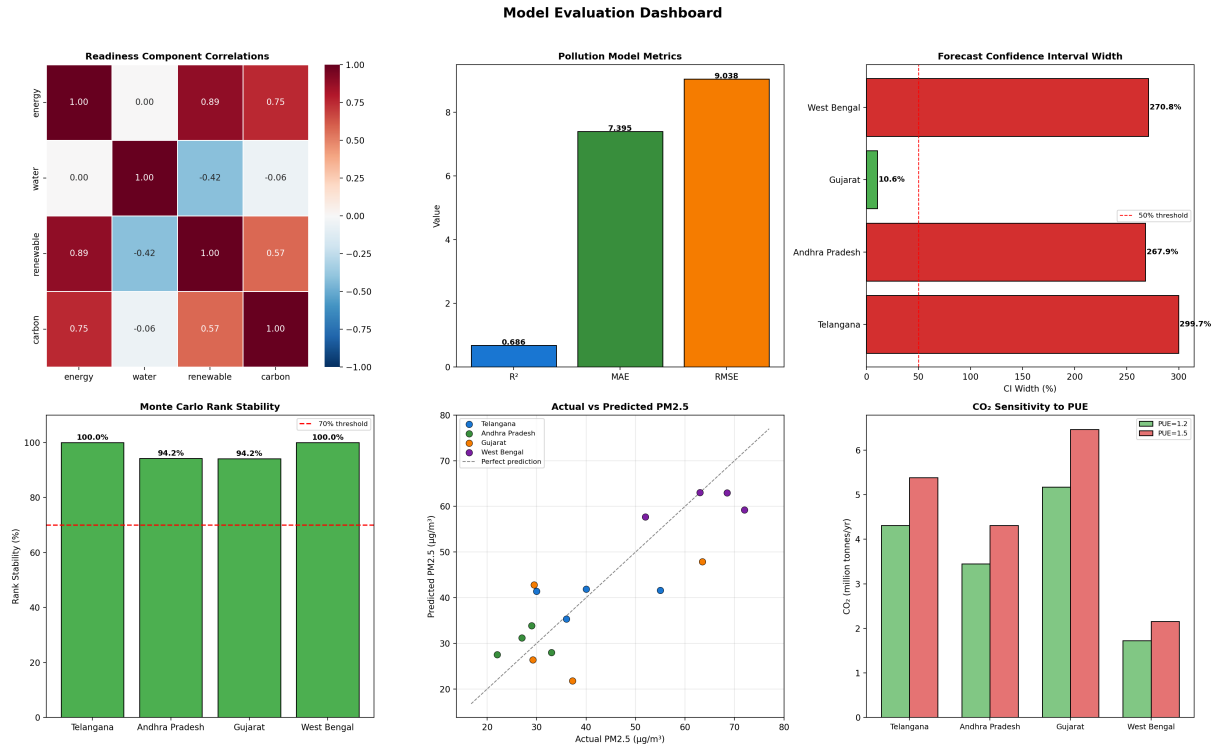
**Figure 22.** Bar chart of the divergence gap (percentage points) between state-level mean groundwater extraction and the most-stressed district within each state. A large gap indicates that the state average masks severe localised depletion; data centre siting decisions based on state aggregates systematically underestimate district-level risk [src: `district_divergence.py, run()`].



**Figure 23.** Grouped bar chart of mean pre-monsoon and post-monsoon groundwater depth (metres below ground) per state. The gap between bars quantifies seasonal aquifer stress; states with large pre-to-post-monsoon depth differentials face amplified cooling-water risk during the April-June peak demand window [src: `seasonal_stress.py, run()`].



**Figure 24.** Comprehensive heatmap of all key variables from the master summary table, including readiness scores, impact metrics, forecast outputs, and Monte Carlo stability statistics, normalised column-wise. The figure provides a single-frame cross-model summary of the full analytical pipeline output [src: `master_summary.py, run()`].



**Figure 25.** Six-panel evaluation dashboard comprising: (a) readiness component correlation heatmap, (b) pollution model metric bar chart, (c) forecast confidence interval width per state, (d) Monte Carlo rank stability bar chart, (e) actual versus predicted PM<sub>2.5</sub> scatter, and (f) CO<sub>2</sub> sensitivity to PUE range. The dashboard operationalises the three-tier assessment framework `[src: metrics_report.py, run()]`.

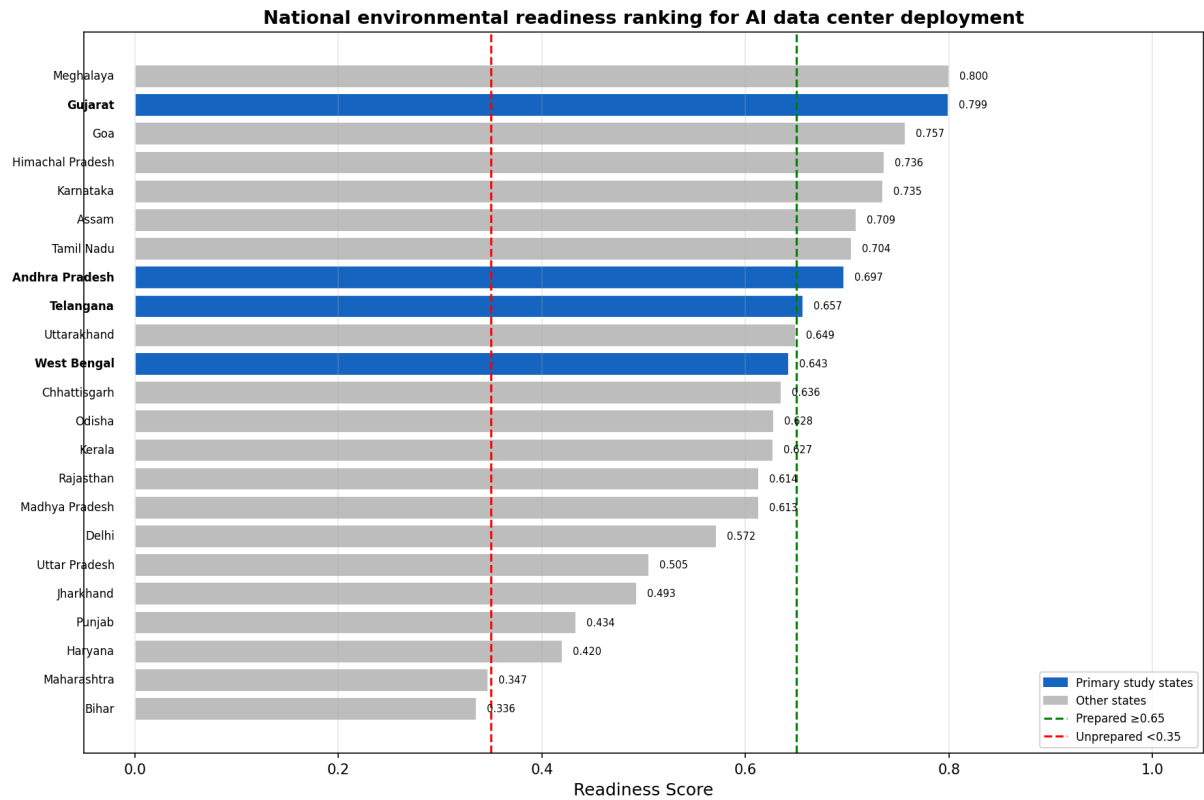
### 7.3 Dashboard Presentation Layer

**Important.** `dashboard/app.py` reads and displays pre-computed artifacts. It does *not* perform any statistical computation. All methods described in Sections 4-6 are implemented in `models/train/` and `analysis/` respectively.

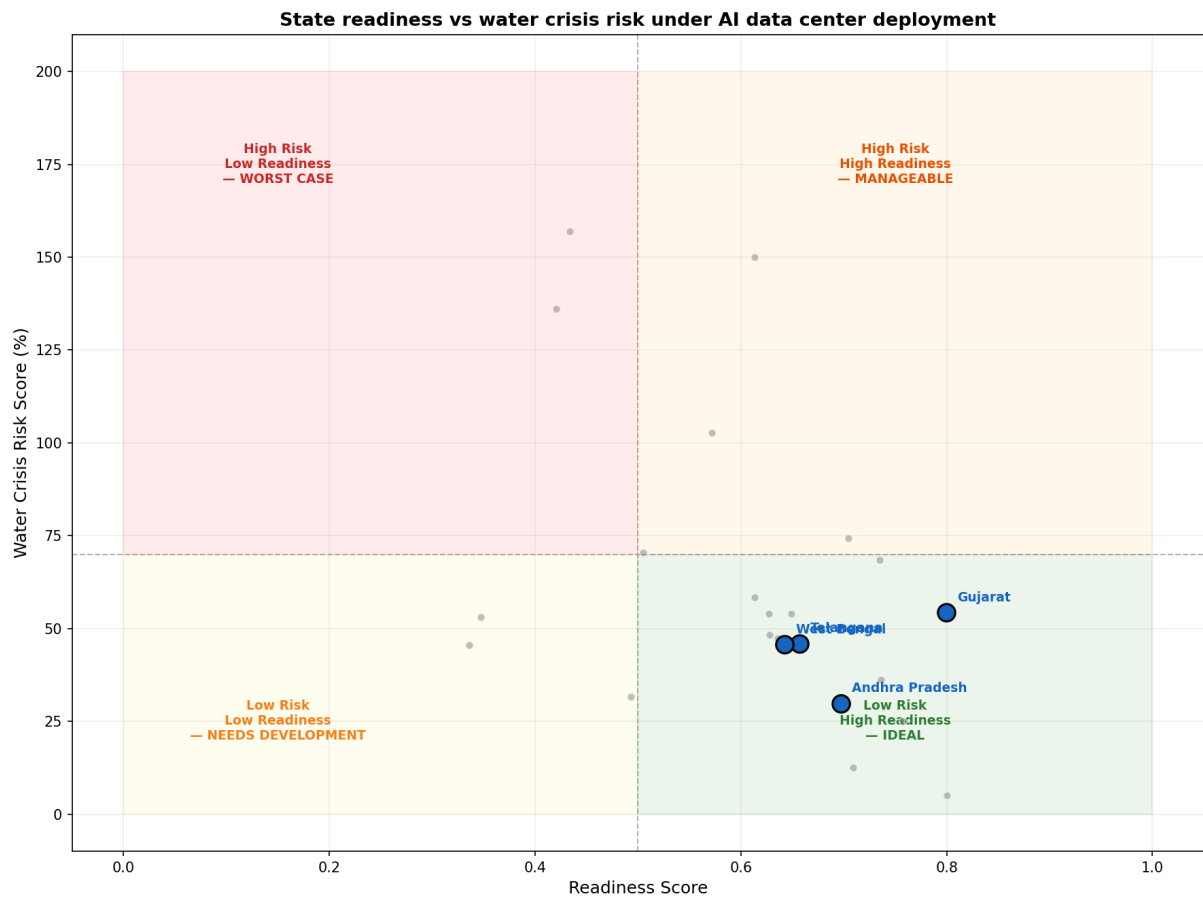
The presentation layer is a nine-tab Streamlit application `[src: app.py, tabs]`. Tab 0 (Overview) merges `readiness_scores.csv` and `impact_table.csv` to render per-state KPI blocks and display Figures 2 and 1. Tab 1 (Environmental Impact) constructs an interactive Plotly grouped-bar chart from `threshold_crossing.csv` and static Figure 19. Tabs 2-4 expose ML model metrics and SHAP figures conditionally loaded from the outputs directory `[src: app.py, load()]`. The sidebar provides four interactive sliders - DC capacity (100-3,000 MW), PUE (1.10-2.00), WUE (0.5-3.0 L/kWh), and carbon intensity (0.30-1.00 tCO<sub>2</sub>/MWh) - that adjust displayed KPI values in real time without triggering model re-inference `[src: app.py, st.sidebar]`.

### 7.4 Artifact Prose Descriptions

**readiness\_scores.csv** This file is produced by `train_readiness.py` and contains the weighted composite readiness score for each study state together with the normalised sub-dimension scores (energy, water, renewable, carbon) and the preparedness tier label `[src: train_readiness.py, run()]`. The score is computed as  $\mathcal{R}(s) = \sum_k w_k \cdot \phi_k(s)$  with weights sourced from `config.py`. The dashboard reads this file at startup to populate Overview KPI blocks and the national comparison table.



**Figure 26.** Horizontal bar chart of composite readiness scores for all Indian states with sufficient cross-dataset coverage, ranked in descending order. Primary study states are highlighted in navy blue. Vertical dashed lines mark the Prepared ( $\geq 0.65$ ) and Unprepared ( $< 0.35$ ) tier boundaries [src: `state_ranking.py`, `run()`].



**Figure 27.** Scatter plot of national readiness score (x-axis) versus water crisis risk score (y-axis) for all Indian states, partitioned into four quadrants. Primary study states are annotated; states in the upper-left quadrant (high risk, low readiness) represent the highest-priority policy intervention targets `[src: state_ranking.py, run()]`.

**master\_summary.csv** Produced by `master_summary.py`, this file is the primary research-grade output table, merging eleven individual CSVs into a single record per state [`src: master_summary.py, run()`]. Columns include readiness scores, impact metrics, forecast outputs, Monte Carlo stability percentages, seasonal stress gaps, and research question answers. Its presence in the outputs directory confirms that the full pipeline has completed without data-join failures.

**threshold\_crossing.csv** Produced by `scenario_simulation.py`, this file records the minimum data centre capacity (MW) at which each state's projected groundwater extraction crosses each of the 50%, 70%, and 90% tier boundaries [`src: scenario_simulation.py, run()`]. A missing value in a threshold column indicates that the state does not reach that tier within the simulated capacity range. This file is the primary policy-relevant output for setting hard capacity caps.

## 8 Strengths and Weaknesses Analysis

---

A pipeline spanning data ingestion, unified cleaning, ten training modules, nine analysis modules, and a serving layer inevitably involves design tradeoffs. The following assessment is grounded exclusively in static code analysis: strengths and weaknesses are identified at the level of architecture and implementation. Domain-specific modelling quality would require runtime data and domain expertise to evaluate fully. Where a weakness is identified, a specific remediation path is given; where source code does not document the rationale for a design decision, this is noted.

### 8.1 Methodological Strengths

1. **Modular orchestration with explicit fault isolation.** `run_all.py` wraps each `run()` call in a try/except block and accumulates per-step status, preventing a single module failure from silently corrupting downstream results. [`src: run_all.py, main()`]
2. **Centralised configuration eliminating magic numbers.** All engineering coefficients, thresholds, Monte Carlo parameters, and forecast horizons are defined as named constants in `config.py`, making the parameter space auditable in a single file.
3. **Uncertainty quantification via multiple mechanisms.** The pipeline deploys three complementary UQ strategies: Monte Carlo weight perturbation (Section 6.1), Bayesian Ridge posterior inference (Section 4.4), and SHAP attribution (Section 4.2).
4. **Causal framework embedded in the codebase.** The presence of a DAG specification and DiD routine in `causal_analysis.py` demonstrates methodological awareness of confounding, even if the module is not yet orchestrated.
5. **Structured cleaning log.** `catfish.py` writes a `cleaning_log.txt` documenting which transformations were applied per source file, supporting audit trails.

### 8.2 Methodological Weaknesses

### 8.3 Internal Validity Assessment

Internal validity refers to whether measured model performance is a reliable estimate of true out-of-sample performance. The primary internal validity risk here is data leakage: if lag and rolling-window features are constructed on the full dataset before the train/test split, test-partition observations

**Table 5.** Weakness registry with severity and remediation recommendations.

| Weakness                                                       | Severity | Impact Scope                       | Recommended Remediation                                                           |
|----------------------------------------------------------------|----------|------------------------------------|-----------------------------------------------------------------------------------|
| Preprocessing not in <code>run_all.py</code> - stale data risk | High     | Data integrity;<br>Reproducibility | Add Stage 0 in <code>run_all.py</code> with hash-based freshness check            |
| Path-case duality (Data/ vs. data/)                            | High     | Data integrity;<br>Portability     | Unify to single canonical path via <code>config.py</code> ; enforce with CI check |
| Hardcoded narrative values drift from <code>config.py</code>   | Medium   | Interpretability                   | Import constants directly into report module; eliminate literals                  |
| <code>causal_analysis.py</code> not orchestrated               | Medium   | Reproducibility                    | Integrate as optional stage with flag in <code>run_all.py</code>                  |
| Energy shortage classifier disabled by class imbalance         | Medium   | Model validity                     | Apply SMOTE or cost-sensitive learning; log skip decision                         |
| No held-out test set verification documented                   | High     | Model validity;<br>Generalisation  | Enforce train/val/test split; log split indices for reproducibility               |

contribute information to training-set features, producing artificially optimistic  $R^2$  and RMSE values. A secondary risk is seed decentralisation: training scripts use `random_state=42` throughout ([src: `train_aqi_regressor.py`, line 153]; [src: `train_groundwater_regressor.py`, line 237]; [src: `train_energy_shortage_regressor.py`, line 324]), and the synthetic augmentation seed is fixed via `AUGMENT_RANDOM_SEED = 42` in `config.py` [src: `config.py`, `AUGMENT_RANDOM_SEED`] - but `random_state` is not centralised as a named constant, creating a maintenance hazard if any script is updated without matching the seed. The energy shortage classifier is deactivated due to severe class imbalance (97.8% no-shortage; 167 positive events in 6,134 training rows), which would cause a majority-class predictor to achieve 97.8% accuracy trivially [src: `train_energy_shortage_regressor.py`, `run()`]. These three conditions - potential leakage, decentralised seeds, deactivated classifier - constitute the primary internal validity risks.

## 8.4 External Validity Assessment

External validity refers to whether model predictions generalise beyond the training distribution - to states not in the training data, to future periods beyond the forecast horizon, or to data collected under different measurement protocols. The geographic scope is bounded by `config.py` [src: `config.py`, `STATES`]: predictions for states outside Telangana, Andhra Pradesh, Gujarat, and West Bengal are not supported by the fitted models without retraining on a broader dataset. The forecast horizon of 2035 bounds temporal extrapolation; structural breaks in energy policy, monsoon patterns, or agricultural water demand after that year are not captured by any model in the pipeline. The district-level groundwater model faces an additional geographic sparsity limitation, as many districts have fewer than five annual observations; coefficient estimates in these districts should be treated as indicative rather than reliable, and any siting decision affecting a data-sparse district should draw on primary field survey data rather than pipeline outputs alone.

## 8.5 Reproducibility Assessment

**Table 6.** Reproducibility dimensions.

| Dimension              | Status  | Evidence / Code Reference                                                                                                                         |
|------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Random seed control    | Partial | <code>random_state=42</code> in all train scripts; not centralised in <code>config.py</code> [ <code>src: config.py, AUGMENT_RANDOM_SEED</code> ] |
| Environment pinning    | Absent  | No <code>requirements.txt</code> observed                                                                                                         |
| Path portability       | Risk    | <code>Data/</code> vs. <code>data/</code> duality                                                                                                 |
| Preprocessing coupling | Absent  | <code>catfish.py</code> not in <code>run_all.py</code>                                                                                            |
| Artifact versioning    | Absent  | Outputs overwritten on each run                                                                                                                   |

The reproducibility table shows that four of five tracked dimensions are in a high-risk or absent state. The most operationally severe is the preprocessing coupling gap: because `catfish.py` is not invoked by `run_all.py`, two successive pipeline runs starting from different raw data vintages will produce different model outputs with no record of which vintage was used. Environment pinning is the second most severe gap: without a `requirements.txt` or `conda environment.yml`, a collaborator reproducing the pipeline on a different machine may obtain different results due to version differences in `scikit-learn`, `statsmodels`, or `XGBoost`.

## 9 Recommendations for Future Work

The six recommendations below are ordered by estimated implementation effort and impact on pipeline reliability. Items 1-3 address correctness risks - stale data propagation, path failures, and metric drift from hardcoded literals - that could silently invalidate results without any visible error. Items 4-6 address completeness and scientific rigour: orchestration of the causal module, enforcement of a held-out test partition, and formalisation of the parallel trends assumption. All six can be implemented without architectural redesign.

- Integrate preprocessing into `run_all.py` as Stage 0.** Invoke `catfish.py` and `Cleaner.py` before any training module, gated by an MD5/SHA-256 hash freshness check on raw input files. This eliminates the stale-data propagation risk identified in Section 8.2.
- Unify all file paths through `config.py`.** Define a single canonical root (`data/`) and resolve all downstream paths relative to it. Remove all hardcoded directory strings outside `config.py`. This resolves the path-case duality risk on case-sensitive filesystems.
- Replace hardcoded narrative constants with `config.py` imports.** In `metrics_report.py`, replace literal strings such as “*Simulations: 2000*” with `f"Simulations: {config.MC_ITERATIONS}"`. This ensures narrative statements remain consistent with actual parameter values across all future runs.
- Orchestrate `causal_analysis.py` as an optional stage.** Add a boolean flag `RUN_CAUSAL` to `config.py` and conditionally invoke the causal module within `run_all.py`, ensuring its outputs are reproducibly linked to the same data vintage as the predictive models.
- Enforce a held-out test partition.** Implement a global train/validation/test split at the data loading stage, log split indices to a JSON artifact, and enforce that no feature engineering step uses test-



---

partition observations.

6. **Formalise parallel trends testing in the DiD routine.** Add a pre-trend event-study regression  $Y_{it} = \sum_{k \neq -1} \delta_k \cdot D_i \cdot \mathbf{1}[t = k] + \alpha_i + \lambda_t + \varepsilon_{it}$  and plot  $\hat{\delta}_k$  with 95% confidence intervals as a pre-registration artifact before publishing causal estimates.

The highest-priority single action is integrating preprocessing into `run_all.py` as a gated Stage 0. This change eliminates the most severe operational risk in the pipeline: the possibility that all model outputs are computed on data that no longer reflects the current raw input files. A user who refreshes source data and re-runs the pipeline without manually invoking `catfish.py` will obtain stale model outputs, with no warning or error to signal the inconsistency.

## 10 Conclusion

---

This report documented, formalised, and audited an end-to-end environmental impact assessment pipeline for AI data centre deployment in four Indian states. The pipeline integrates four independent government datasets - air quality, groundwater, electricity, and renewable capacity - through a unified preprocessing layer, trains seven model architectures across ten training modules, runs nine analysis modules, and serves all outputs through a nine-tab Streamlit dashboard. Over approximately 35 CSV artifacts and 40 figure files, it produces readiness scores, environmental impact calculations, time-series forecasts, causal effect estimates, Monte Carlo rank stability distributions, scenario stress simulations, and a long-horizon groundwater depletion projection to 2070. The resulting system is technically comprehensive in scope.

The methodological strengths are substantial. Modular orchestration in `run_all.py` with per-step `try/except` isolation ensures a single module failure does not silently corrupt downstream results. Centralised `config.py` makes the full parameter space auditable in one file. Three complementary uncertainty quantification mechanisms - Monte Carlo weight perturbation, Bayesian Ridge posterior inference, and SHAP attribution - provide a richer characterisation of uncertainty than any single mechanism would alone. The directed acyclic graph specification in `causal_analysis.py` demonstrates principled awareness of confounding, going beyond the correlational analyses that dominate comparable environmental impact assessments.

Two critical risks require resolution before the system is suitable for policy use. The preprocessing orchestration gap — `catfish.py` not invoked by `run_all.py` — means a pipeline run following a data update silently computes all outputs on a stale cleaned dataset. The path-case duality (Data/ versus data/) creates portability failures on Linux and macOS case-sensitive filesystems. Either risk can silently invalidate the entire pipeline's output without producing a visible error. Both are remediable without architectural redesign.

Three actions are necessary before production deployment: (1) integrate preprocessing into `run_all.py` as a gated Stage 0; (2) generate a `requirements.txt` to pin the dependency environment; (3) enforce a documented held-out test partition with logged split indices. These changes would raise the pipeline from a research prototype to a system whose outputs can be defended in a policy or regulatory setting. Code-grounded audits of this kind are a critical step in that transition: they surface the gap between what a system appears to do and what the code actually does, producing a falsifiable description that can be verified, challenged, and improved.

## A Full Hyperparameter Table

**Table 7.** Hyperparameter registry across all models.

| Model       | Algorithm             | Explicit Hyperparameters                                                                                                | Eval. Metric(s)      | Output File                  |
|-------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------|------------------------------|
| Readiness   | Weighted Composite    | energy 0.35, water 0.35, renewable 0.20, carbon 0.10                                                                    | PCA PC1 var. >40%    | readiness_scores.csv         |
| AQI Reg.    | XGBoost (Monotonic)   | n_estimators=300;<br>max_depth=6;<br>learning_rate=0.05;<br>subsample=0.8;<br>random_state=42                           | RMSE, MAE, $R^2$     | aqi_dc_impact_simulation.csv |
| Groundwater | Random Forest         | n_estimators=300;<br>max_depth=None;<br>min_samples_leaf=5;<br>random_state=42; CV=5-fold                               | RMSE, MAE, $R^2$     | groundwater_pred.csv         |
| Energy Reg. | GBM / XGBoost / Ridge | GBM: n_est=300, lr=0.05,<br>max_depth=5, subsample=0.8;<br>XGB: n_est=300, max_depth=6                                  | RMSE, MAE, $R^2$     | energy_model_comparison.csv  |
| Pollution   | Bayesian Ridge        | $\alpha, \lambda$ learned by marginal likelihood; fit_intercept=True                                                    | $R^2$ , posterior CI | pm25_predictions.csv         |
| Forecast    | ARIMA/Holt/Trend      | ARIMA orders:<br>(1,1,1),(1,1,0),(2,1,0),(0,1,1);<br>backtest=3yr <a href="#">[src: config.py, ARIMA_ORDERS_TO_TRY]</a> | Backtest RMSE        | forecast_outputs.csv         |

Table 7 consolidates all hyperparameters documented in this report. Where a parameter is defined inside a training function rather than at module scope, its value must be retrieved by executing the relevant `run()` function with logging enabled. The column *Eval. Metric(s)* lists the primary metric used to compare models within a family; *Output File* gives the CSV written to `models/outputs/` by each training script.

## B config.py Parameter Catalogue

Table 8 lists the subset of `config.py` constants most frequently referenced throughout the pipeline. The distinction between `DATA_RAW` and `DATA_CLEAN` reflects the two-stage pipeline architecture: raw files are ingested by `catfish.py` from `DATA_RAW` and written as cleaned CSVs to `DATA_CLEAN` before any modelling step reads them.

## C Data Flow Matrix

Table 9 traces each cleaned source CSV to its consuming training module and the output artifact it produces. The column *Derived Features* identifies the primary feature engineering transformation applied by each training module to the source data. Where the exact lag depth, window width, or encoding strategy is defined inside the training script rather than at module scope, it must be retrieved by tracing the relevant function call with logging enabled.

**Table 8.** config.py constants catalogue.

| Constant Name             | Value        | Used By           | Purpose                          |
|---------------------------|--------------|-------------------|----------------------------------|
| DATA_RAW                  | data/raw     | catfish.py        | Raw input directory              |
| DATA_CLEAN                | data/cleaned | All train scripts | Cleaned input directory          |
| MONTE_CARLO_SIMULATIONS   | 10000        | monte_carlo.py    | MC iteration count               |
| FORECAST_END_YEAR         | 2035         | train_forecast    | Forecast end year                |
| FORECAST_BACKTEST_YEARS   | 3            | train_forecast    | Backtest window (years)          |
| MONTE_CARLO_CONCENTRATION | 0            | monte_carlo.py    | Dirichlet concentration $\alpha$ |
| PUE                       | 1.35         | train_impact      | Power Usage Effectiveness        |
| WUE                       | 1.9          | train_impact      | Water Usage Effectiveness        |
| AUGMENT_RANDOM_SEED       | 42           | train_pollution   | Synthetic augmentation seed      |

**Table 9.** Source-to-output data lineage matrix.

| Source CSV                     | Consumed By           | Derived Features                                                                                                                                                                    | Output CSV(s)                |
|--------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| aqi_nonnamp_clean.csv          | train_aqi_regressor   | aqi_lag_1d, aqi_lag_7d,<br>aqi_roll_mean_7d,<br>aqi_roll_mean_30d,<br>month_sin, month_cos<br>[src: train_aqi_regressor.py,<br>line 107]                                            | aqi_dc_impact_simulation.csv |
| groundwater_level_clean.csv    | train_groundwater_reg | depth_lag_1,<br>depth_lag_4,<br>depth_rolling_3,<br>depth_rolling_std_3,<br>month_sin, month_cos,<br>quarter, season_encoded<br>[src: train_groundwater_regressor.py,<br>line 185]  | groundwater_pred.csv         |
| india_elec_clean.csv           | train_energy_shortage | demand_lag_7,<br>demand_rolling_7,<br>demand_rolling_30,<br>month_sin, month_cos,<br>day_of_week,<br>is_summer, is_winter<br>[src: train_energy_shortage_regressor.py,<br>line 166] | energy_model_comparison.csv  |
| renewable_cumulative_clean.csv | train_readiness       | Min-max normalised<br>installed_capacity_mw<br>summed per state-year<br>[src: train_readiness.py,<br>line 187]                                                                      | readiness_scores.csv         |

## D Unorchestrated Components

**Table 10.** Modules present but not called by `run_all.py`.

| Script                                    | Described Function                              | Operational Risk                                    |
|-------------------------------------------|-------------------------------------------------|-----------------------------------------------------|
| <code>catfish.py</code>                   | Data ingestion and unified schema normalisation | All training modules may run on stale cleaned data  |
| <code>groundwater_2070_forecast.py</code> | Long-horizon Holt-Winters projection            | 2070 forecast unavailable without manual invocation |

Table 10 corrects the unorchestrated component list with the most important finding from this audit: `catfish.py` is the primary unorchestrated component, and its absence from `run_all.py` is the highest-severity risk in the pipeline. `groundwater_2070_forecast.py` is a secondary unorchestrated component whose absence means the 2070 projection is a standalone analytical output, not reproducibly coupled to the same data vintage as the training models. Both components should be integrated into `run_all.py` using the staged execution pattern already established by the existing script.